

Continuous Queries as Maintained Answers

Exact Cuts, Exact Seams and Lossless Cutover

in a Single-Node Streaming SQL Engine

Ashutosh Sinha

Independent Researcher

`ajsinha@gmail.com`

Abstract

An application that needs the current answer to a question over changing data usually asks again: it polls a database, reruns a batch job, or rebuilds a cache on a timer. Between two askings the answer is stale, and each asking pays for the whole question rather than for what changed. A continuous query inverts the arrangement. The question is registered once, the engine keeps its answer current as the data changes, and reading the answer is a lookup. The inversion is old; what makes it hard to trust is not the incremental arithmetic, which is well understood, but the *hand-overs*: the moments at which one stream of records has to take over from another without losing a record or counting one twice. A subscriber joining a running view, a restart resuming from a checkpoint, a query attaching to a source reader that other queries are already sharing, and a new version of a query taking the name from the old one are four such hand-overs, and each is a place where a system can be almost right in a way that nothing reports.

We describe how PRAVAHA, a single-node, embeddable continuous-SQL engine written in Java 21, makes each of the four exact, and we argue that the four are one problem with one answer: *a seam is a position, not a moment*. Each hand-over is made at a point in the input named by source positions that are totally ordered and replayable; the positions and the state they describe are read together under a freeze; and where equality at the seam cannot be established the engine refuses rather than approximates. From that we state and prove, under assumptions we list, four results: a subscription started from a snapshot receives the view at some commit and then every later commit exactly once (Theorem 4.6); a checkpoint is one consistent cut, so restored state reflects each input record exactly once and a two-phase sink receives each change exactly once (Theorems 5.5 and 5.9); a reader shared by queries at different positions hands each query each record exactly once and in order, whether the query is attached, catching up or waiting (Theorem 6.6); and a blue/green replacement cuts over at a position both versions have consumed exactly, so a reader sees the old answer up to the seam and the new one after it, and a rollback within the retention window is the same operation reversed (Theorem 7.3). Underneath these sit the model the engine computes in — rows with integer weights, after DBSP — and its operator algebra, whose laws we state with proofs and with an honest account of where the running engine is and is not held to them.

Every formal claim is marked with what carries it in the implementation: a named test that runs, a test that covers part of it, an argument from the code with no test, or nothing. The markers are not uniformly favourable. The reference algebra is tested as stated but is imported by no module of the running engine; the served view has an open finding against one of its invariants; and the engine is one node, with clustering built as libraries that no node consumes. Section 11 collects the markers, Section 12 reports only figures present in the repository together with the conditions they were taken under, and Section 14 states what the implementation does not establish.

Contents

1	Introduction	3
1.1	Asking again, and being answered	3
1.2	Where continuous answers go wrong: four hand-overs	4
1.3	The thesis: a seam is a position	4
1.4	Contributions	5
1.5	What is borrowed, and what is not claimed	6
1.6	How to read this paper, and how to check it	6
2	The objects, in engineering terms	6
3	The model: rows with weights	8
3.1	Z-sets	8
3.2	Queries and their incremental forms	8
3.3	What the running engine is held to	10
3.4	The served view	11
3.5	Event time, watermarks, windows and lateness	12
3.6	State that is bounded, or refused	13
4	Reading a maintained answer: commits and subscriptions	14
4.1	Commits	14
4.2	Subscribing from a snapshot	15
5	Exactly once: a checkpoint is one cut	16
5.1	The setting	16
5.2	The aligned cut	17
5.3	Output: two-phase sinks	18
6	Sharing a reader at an exact seam	20
6.1	Why the earlier sharing could not be exact	20
6.2	Two declarations	20
6.3	Three states	21
7	Lossless replacement: a position both versions have consumed	23
7.1	The splice	23
7.2	The cutover	24
7.3	The rollback window	25
8	Lanes: isolation against memory	26
8.1	Confinement	26
8.2	Own, shared, auto and dedicated	26
8.3	Moving a query is a replacement	28
9	Serving the answer	29
9.1	Consistency of a read	29
9.2	Access paths change cost, never answers	29
9.3	An equality index over a column outside the key	29
10	Security, identity and tenancy, as they affect the model	31
11	The register: what carries each claim	32

12 Evaluation: what has been measured, and under what conditions	34
12.1 The requirement, and the machine	34
12.2 Throughput gates	35
12.3 Generated against interpreted code	35
12.4 Nexmark coverage	35
12.5 What a query costs	36
12.6 Subscribers, state beyond memory, and instrumentation	36
12.7 What has not been measured	36
13 Related work	37
14 Limitations, what is not built, and future work	38
14.1 What the implementation does not establish	38
14.2 How the account could be shown wrong	39
14.3 Future work	40
15 Conclusion	40
A Where the code is	42

1 Introduction

1.1 Asking again, and being answered

Consider a payments service that must show, for every card, the number and total value of transactions in the last ten seconds, the customer’s tier looked up as of each transaction’s time, and only completed transactions counted. The conventional arrangement stores the transactions in a database and asks the question when somebody needs the answer: a dashboard polls every few seconds, a fraud rule issues the query per decision, a nightly job recomputes a summary table. Three things follow. The answer is stale between askings by up to the polling interval. Each asking pays for the whole question, so the cost follows the size of the data and the number of askers rather than the rate at which the data changes. And the questions are anonymous to the database: ten desks asking the same thing cost ten executions.

A *continuous query* inverts this. The question is registered once, with a name and the columns its answer is keyed by:

```
CREATE CONTINUOUS QUERY txn_volume KEYED BY (window_end, user_id) AS
  SELECT TUMBLE_END(event_time, INTERVAL '10' SECOND) AS window_end,
         t.user_id, p.tier, COUNT(*) AS txn_count, SUM(t.amount) AS total_volume
  FROM txn_stream AS t
  LEFT JOIN user_profile FOR SYSTEM_TIME AS OF t.event_time AS p
    ON t.user_id = p.user_id
  WHERE t.status = 'COMPLETED'
  GROUP BY TUMBLE(t.event_time, INTERVAL '10' SECOND), t.user_id, p.tier
```

The engine then keeps the answer current as the data changes: it reads each change once, applies it to the state the question needs, and publishes the difference it makes to the answer. Reading the answer is a lookup by key. A consumer that wants the changes rather than the answer subscribes and receives one batch per commit, each row carrying a weight, +1 for a row appearing and −1 for a row withdrawn, so a window corrected by a late record arrives as a retraction of the old answer and the new one. The slogan PRAVAHA uses for this is *ask once, answer always*.

None of this is new as an idea. Continuous queries over append-only data go back to Terry et al. [1]; incremental view maintenance is a mature literature [2, 3, 4, 5, 6]; and DBSP [7] gives a small, general algebra for incrementalising a query language, which PRAVAHA follows. What this paper is about is what those results assume and a running system has to supply.

1.2 Where continuous answers go wrong: four hand-overs

The incremental arithmetic of a maintained view is the part that is easiest to get right, because it can be stated as an equation and tested against recomputation. The failures practitioners meet are elsewhere. They are at the moments where *one stream of records takes over from another*, and a system can be almost right at each of them in a way that no error reports.

H1: a subscriber joins a running view. A client that keeps a copy of the view reads the current rows and then subscribes to changes — or subscribes and then reads. Whichever order it chooses, a commit can land between the two steps, and the client’s copy is then missing that commit or holding it twice, for ever, with nothing in either stream to say so.

H2: a restart resumes from a checkpoint. The engine records its operator state and the position it had reached in each source, and after a crash restores the state and resumes reading from the positions. If the positions and the state do not describe the same point in the input — the positions read a moment after the state was copied, say — the restart either skips the rows in between or counts them twice. The answer is then merely a little wrong, which is the most expensive kind of wrong.

H3: a query joins a reader others are sharing. A thousand queries over one topic should not read it a thousand times, so a reader is shared. A query registered later, or resumed after a pause, or restored from an older checkpoint, is at a different position from the shared reader, and has to be brought to it. Catching it up by reading “to the end” reads past the shared reader and delivers the overlap twice; attaching it at once skips what it missed.

H4: a query is replaced by a new version. Changing a registered query’s SQL should not take its answer away while the new version rebuilds its state, so the new version runs beside the old one and takes the name when it has caught up. “Caught up” at a wall-clock instant is not a definition: two queries running at slightly different speeds swapped at 14:00 have consumed different input, and the records between them are in neither answer or in both.

1.3 The thesis: a seam is a position

Each of H1–H4 is a *seam*: a point at which records stop coming from one place and start coming from another. Our thesis is that the four are one problem, and that it has one answer with three parts.

1. **The seam is named by a position, not a moment, a count or a guess.** A position is a token a source handed out naming a point in its log, and positions within a partition are totally ordered and replayable: a reader opened at a position delivers exactly the records after it. A record count is not a position (Kafka’s offsets have gaps), a wall-clock time is not a position, and “the reader looks idle” is not a position.
2. **Positions and state are read together, under a freeze.** Where a seam depends on what a computation has consumed, the sources are held between rows while the positions are read and a marker is placed in the computation’s input at exactly that point; the state

is taken at the marker. What the positions exclude and what the state includes are then the same rows.

3. **Where equality at the seam cannot be established, the engine refuses.** A history that runs out before reaching its seam stops the backfill; two versions that cannot be brought to equal positions within a budget are not cut over; a catch-up that makes no progress toward its seam stops the group; a subscriber that has fallen too far behind is ended rather than written past the gap. Each refusal carries a named code.

H1 is solved by making “the view at a commit” and “every commit after it” one step under the lock that orders commits (Section 4). H2 is solved by the aligned checkpoint of Section 5. H3 is solved by two declarations a source can make — its positions are ordered, and its readers can stop at a position — and a three-state membership protocol (Section 6). H4 is solved by splicing a replay of the history onto the live stream at the exact position the running version has reached, and cutting over only when both versions’ positions are equal token for token (Section 7). The administrative operation of moving a query between a shared lane and a lane of its own (Section 8) turns out to be H4 with the SQL unchanged, and inherits its guarantee.

1.4 Contributions

Stated precisely, and with the evidence each rests on named in the section that states it:

1. The observation that four hand-overs a continuous-query engine must perform — subscription from a snapshot, recovery from a checkpoint, joining a shared source reader, and replacing a query — share one structure, and a single design discipline (ordered, replayable positions; a freeze; refusal on inequality) that makes all four exact (this section and Sections 4 to 7).
2. A snapshot-then-commits subscription theorem for a view whose commits are staged for an audience fixed at the commit’s first batch (Theorem 4.6), with the assumptions under which it holds and the ones under which it does not (subscriber overflow, listener failure).
3. A statement and proof sketch of exactly-once state from an aligned, single-lane checkpoint (Theorem 5.5) and of exactly-once output to a two-phase sink prepared at the same marker (Theorem 5.9), with the assumptions — replayable sources, deterministic operators, idempotent commit, a single lane — stated as assumptions rather than folded into the result.
4. An exact-seam protocol for sharing one reader of an ordered source among queries at different positions, with a proof that each member receives each record exactly once and in order in each of the attached, catching-up and waiting states (Theorem 6.6). We believe the combination of an order declaration and a bounded read (`pollBefore`) as the contract a source plugin offers, with membership states derived from comparing positions, is new in this form.
5. A lossless blue/green replacement for a continuous query: a history reader spliced onto the live stream at an offset the running version has read to, and a cutover taken only at equal positions with both feeds stopped (Lemma 7.1 and theorem 7.3); and the observation that keeping the replaced version fed for a retention window makes rollback the same alignment reversed (Proposition 7.4).
6. An account of lanes — the unit that owns a thread slot, an inbox and an arena — as a choice between isolation and memory, with fate-sharing stated as a property rather than discovered as an incident (Proposition 8.4), and lane rebalancing reduced to replacement (Corollary 8.5).

7. For serving: the rule that an access path may change what a read costs and never what it answers, and the exact condition under which an equality index over a non-key column is sound — stored-value equality must refine the filter’s equality (Proposition 9.3) — which is why `FLOAT`, `DECIMAL` and `BYTES` columns are refused.
8. A register (Section 11) that states, for every formal claim, whether a named test in the repository carries it, carries part of it, or carries none of it.

1.5 What is borrowed, and what is not claimed

The weighted-row model, the lift of a query to its incremental form, the bilinear join rule, and the differentiation and integration operators are DBSP’s [7], which in turn formalises ideas from differential dataflow [8] and the ring-of-databases view of incremental evaluation [5]; the signed-multiplicity relations are Green, Karvounarakis and Tannen’s $\mathbb{Z}$ -relations [10, 11]. Aligned checkpoints are Chandy and Lamport’s consistent snapshot [12] in the form Flink made standard [13, 15]. Watermarks, allowed lateness and the retraction of a corrected window are the Dataflow model’s [16, 17]. Two-phase output to external systems follows the same lineage as Kafka’s transactional producer and Flink’s two-phase commit sinks [22]. Whole-stage code generation is Neumann’s [27]. Shared scans with late joiners have a long history in databases [28, 29].

We do not claim a new incremental algebra, a faster engine, or a distributed system. PRAVAHA runs on one node; its clustering primitives are built as libraries and consumed by no node (Section 14). We make no performance claim beyond the figures the repository records, with their conditions (Section 12), and those figures were taken on a loaded laptop.

1.6 How to read this paper, and how to check it

Section 2 states the objects in engineering terms before the mathematics begins. Section 3 gives the model: weighted rows, the operator algebra, the served view, event time. Section 4 is H1, Section 5 is H2, Section 6 is H3, Section 7 is H4. Section 8 is the execution substrate and what it isolates; Section 9 is reading; Section 10 is security and tenancy as they affect the model. Section 11 collects every claim’s marker, Section 12 the measurements, Section 13 related work, and Section 14 limitations and future work.

The mathematics is elementary and the vocabulary is unevenly shared between the audiences this is written for. Every formal statement is therefore followed by a shaded *plain terms* gloss and, where one earns its place, a worked example. A reader who wants the argument without the notation can follow the shaded material alone.

Every formal claim carries one of five markers. **Runs** names the test classes and test methods in the PRAVAHA repository that assert it; every name was checked against the tree at the commit this paper was built from. **In part** says which part runs and which does not. **Argued, not tested** says the claim follows from code we cite and no test asserts it. **Reference algebra only** says the claim holds of the module `pravaha-algebra`, which is tested, but not demonstrably of the running engine, for a reason Section 3.3 makes plain. **Not in PRAVAHA** says the system does not carry the construction. The markers are placed where a reader would look for the favourable one, and Section 11 collects them into a table that can be checked against the repository in an afternoon.

2 The objects, in engineering terms

Nothing in this section is a contribution. It is the setting, written in the words a system that implements it has to use, and written first because the results later are statements about exactly these objects and are easy to mishear as statements about something looser.

A stream is a declared name over one or more sources. A stream has a schema, a column that carries *event time* — the moment a row describes, not the moment the engine saw it — an out-of-orderness (how long to wait before calling a period complete) and an allowed lateness (how long after that a row may still correct an answer). Rows reach a stream from a *source binding*: a file, a directory of files, a Delta table, a JDBC table polled by a watermark column, an Aerospike set or Cassandra table scanned, a PostgreSQL or MySQL change feed, or a Kafka topic.

A source is partitioned, and a partition has positions. A source splits into partitions, each read by a *partition reader*. A reader reports a *position*: an opaque token that names the point in the partition after the last record it consumed. The contract every reader keeps is the one a restore relies on: a reader created at a position delivers the records after it. For a file the position is the count of lines consumed; for Kafka it is the partition offset; for a change feed it is a log sequence number or a binlog file and offset. A table scan is different in kind: it reports where its pass began, which does not order the records inside the pass.

A continuous query is a registration, and a computation sits behind it. Registering SQL under a name creates a *computation*: a plan compiled into a pipeline of operators with state. Two registrations whose normalised plans are identical — including the security predicates applied to them, the tenant, the key columns and the retention — are one computation under two names. Registration and subscription are separate objects with separate lifetimes; a computation outlives its subscribers.

A lane is what runs a computation. A lane owns an inbox (a bounded ring of fixed-size off-heap cells), an arena (off-heap bump-pointer slabs) and the operator instances that read from the inbox, and it is driven by exactly one thread at a time from a fixed pool of one thread per core. By default the first 64 queries on a node each own a lane and later ones share a fixed set of lanes; Section 8 is about what that choice buys.

A view is the answer, and it is committed. Each registered query maintains a *served view*: a map from the declared key to the current row. Changes are applied to a pending overlay as batches finish, and a *commit* moves the overlay into the visible map atomically, stamped with a *frontier* — the input position, in event time, the change reflects. A consistent read sees the last commit; a subscriber receives commits whole.

A checkpoint is a cut. A checkpoint records, for one computation, its operator state, the position of every source partition it reads, and the committed contents of its view, all at one point in its input. A restart restores the newest checkpoint and resumes each reader from its recorded position.

A sink is where the answer is written. A registration may name a sink — a Kafka topic, a JDBC table, a Delta or Iceberg table, an Aerospike set, a file — and every commit of its view, retractions included, is written there. A sink declares what it can promise: a transactional sink is prepared at a checkpoint's cut and committed once the checkpoint is durable; an idempotent upsert sink is effectively once; a plain append is at least once.

A replacement is a second computation that takes a name. `CREATE OR REPLACE CONTINUOUS QUERY` starts the new version beside the old one. It replays the source history, splices onto the live stream where the old version stands, and takes the name only at a position

both have consumed exactly. The old version keeps running for a retention window so a rollback is one swap.

In plain terms. Eight words carry the rest of this paper. A *stream* is where rows come from, stamped with the time they describe. A *position* is a bookmark in one partition of a source, and a reader opened at a bookmark reads what comes after it. A *query* is registered once and kept running; its *view* is the answer, updated in *commits*. A *lane* is the worker that runs it. A *checkpoint* saves the work and the bookmarks together. A *sink* is where the answer is also written. A *replacement* is a new version of the query brought up beside the old one. The claim of the paper is that the four risky moments — joining a running view, restarting, sharing a reader, and swapping versions — are all bookmark problems, and that they are exact if and only if the swap is made at a bookmark rather than at a time.

3 The model: rows with weights

3.1 Z-sets

Definition 3.1 (Z-set). Let R be a set of rows. A *Z-set* over R is a function $a: R \rightarrow \mathbb{Z}$ whose support $\text{supp}(a) = \{r \in R : a(r) \neq 0\}$ is finite. Write $\mathbb{Z}[R]$ for the set of Z-sets over R , 0 for the Z-set with empty support, and for $a, b \in \mathbb{Z}[R]$ define $(a + b)(r) = a(r) + b(r)$ and $(-a)(r) = -a(r)$. A *relation* is a Z-set whose weights are all $+1$ on its support; a *change* is any Z-set.

Proposition 3.2 (Z-sets form an abelian group). $(\mathbb{Z}[R], +, 0, -)$ is an abelian group, and a representation that stores only the support — never a row with weight zero — represents each element uniquely.

Proof. Pointwise addition in $\mathbb{Z}$ is associative and commutative with identity 0 and inverse $-a(r)$; the support of a sum is contained in the union of the supports, so it stays finite. Uniqueness: two finite maps from rows to non-zero integers that agree as functions $R \rightarrow \mathbb{Z}$ have the same key set (the support) and the same values on it. $\square$

Runs: `ZSetTest` in `pravaha-algebra`, as generated properties: `additionIsAssociative`, `additionIsCommutative`, `emptyIsTheIdentity`, `everyElementHasAnInverse`, `negationIsAnInvolution`; and the representation invariant as `zeroWeightMeansAbsentAndIsNeverStored` and `scalingByZeroYieldsEmptyRatherThanZeroWeights`. `ZSet.Builder.add` removes a row whose weight sums to zero at the moment it does, so a $+1/-1$ pair cancels inside an operator and never reaches the next.

In plain terms. Every row carries a whole number. $+1$ means “this row is here”, -1 means “take this row away”, and $+3$ means “three copies”. A table is a bag of rows at $+1$; a batch of changes is a bag of rows with mixed signs. They are the same kind of object, so applying a change to a table is addition, and an update is not a third kind of message: it is -1 of the old row plus $+1$ of the new.

3.2 Queries and their incremental forms

Definition 3.3 (Query and incremental form). A *query* is a function $Q: \mathbb{Z}[I] \rightarrow \mathbb{Z}[O]$. An *incremental form* of Q is a function $Q^\Delta: \mathbb{Z}[I] \times \mathbb{Z}[I] \rightarrow \mathbb{Z}[O]$ such that for every state s and every change δ ,

$$Q(s + \delta) = Q(s) + Q^\Delta(\delta, s). \quad (1)$$

Q is *linear* when $Q(a + b) = Q(a) + Q(b)$ for all a, b .

Equation (1) is the contract of the interface `IncrementalQuery` in `pravaha-algebra`, whose `evaluateDelta(delta, state)` is Q^Δ ; `Query` is Q , deliberately just a function from Z-sets to Z-sets so that the lift can be stated over it.

Proposition 3.4 (Linear queries lift without state). *If Q is linear then $Q^\Delta(\delta, s) = Q(\delta)$ is an incremental form of Q that does not read s . The selection $\sigma_p(a)(r) = [p(r)] a(r)$, the projection $\pi_f(a)(o) = \sum_{r:f(r)=o} a(r)$ and the flat-map $\phi_g(a)(o) = \sum_r a(r) \cdot \#\{o \in g(r)\}$ are linear, and the composite of two linear queries is linear.*

Proof. If Q is linear, $Q(s + \delta) = Q(s) + Q(\delta)$, which is (1). Selection: $\sigma_p(a + b)(r) = [p(r)](a(r) + b(r)) = \sigma_p(a)(r) + \sigma_p(b)(r)$. Projection: the sum over a fibre of f distributes over pointwise addition. Flat-map: the same, with each row counted once per output it produces. Composition: $Q_2(Q_1(a + b)) = Q_2(Q_1a + Q_1b) = Q_2Q_1a + Q_2Q_1b$. $\square$

Reference algebra only: `IncrementalOracleTest: filterIsLinearAndSoNeedsNoState, projectIsLinear, flatMapIsLinear, filterThenProjectComposes`; and `ZSetTest: filterDistributesOverAddition, projectionDistributesOverAddition, projectionAddsTheWeightsOfRowsThatMerge`. `Lift.linear` is the lift itself. The oracle is shown to have teeth: `theOracleCatchesADeliberatelyBrokenLift`.

In plain terms. A filter or a projection needs no memory to run incrementally: apply it to the change and you get the change in the result. Nothing has to be stored, which is why a query that only filters and projects costs nothing between rows. Projection adds up the weights of rows that land on the same output, which is exactly what SQL's bag semantics asks for when a column is projected away.

Proposition 3.5 (`DISTINCT` is not linear, and its lift is still correct). *Let $\text{dist}(a)(r) = [a(r) > 0]$. Then dist is not linear, and $\text{dist}^\Delta(\delta, s) = \text{dist}(s + \delta) - \text{dist}(s)$ is an incremental form of it, which emits a row only when its presence changes.*

Proof. Non-linearity: for a single row r , $\text{dist}(\{r:1\} + \{r:1\}) = \{r:1\}$ while $\text{dist}(\{r:1\}) + \text{dist}(\{r:1\}) = \{r:2\}$. The incremental form satisfies (1) by construction: $\text{dist}(s) + (\text{dist}(s + \delta) - \text{dist}(s)) = \text{dist}(s + \delta)$. A row whose weight moves from 3 to 4 is present before and after, so its term cancels and nothing propagates. $\square$

Reference algebra only: `IncrementalOracleTest: distinctIsNotLinearButItsLiftIsStillCorrect`, and `theOracleCatchesAnOffByOneInTheDistinctLift` as the check that the property would fail on the obvious mistake. `Lift.distinctIncremental`.

Proposition 3.6 (Recomputation is always an incremental form). *For any query Q , $Q_{\text{rec}}^\Delta(\delta, s) = Q(s + \delta) - Q(s)$ satisfies (1). It costs $O(|s|)$ per change rather than $O(|\delta|)$.*

Proof. Immediate from the group structure: $Q(s) + Q(s + \delta) - Q(s) = Q(s + \delta)$. $\square$

Reference algebra only: `theRecomputationFallbackIsCorrectForAnyQuery` in `IncrementalOracleTest`; `Lift.byRecomputation`. It is the reference the other lifts are compared against, and the escape hatch for a construct whose incremental form is impractical.

Definition 3.7 (Integration and differentiation). For a sequence of Z-sets $x = (x_0, x_1, \dots)$ define $I(x)_t = \sum_{i \leq t} x_i$ and $D(x)_t = x_t - x_{t-1}$ with $x_{-1} = 0$.

Proposition 3.8 (D and I are inverse). *$D(I(x)) = x$ and $I(D(x)) = x$ for every sequence x .*

Proof. $D(I(x))_t = \sum_{i \leq t} x_i - \sum_{i \leq t-1} x_i = x_t$, and $I(D(x))_t = \sum_{i \leq t} (x_i - x_{i-1})$ telescopes to $x_t - x_{-1} = x_t$. $\square$

Reference algebra only: `ZSetTest: differentiateAndIntegrateAreInverses, integrationAccumulates; Integrate, Differentiate`.

In plain terms. “Integrate” is a running total of changes; its state is the current answer. “Differentiate” turns a sequence of answers back into the changes between them. They undo each other. The practical reading is the one the engine is built on: a maintained view *is* the running total of its changes, so serving the answer is reading that total, not keeping a second copy in step with it.

Theorem 3.9 (The bilinear join rule). *Let $\bowtie: \mathbb{Z}[L] \times \mathbb{Z}[R] \rightarrow \mathbb{Z}[O]$ be the equi-join ($a \bowtie b)(o) = \sum_{\ell, r: k(\ell)=k'(r), c(\ell, r)=o} a(\ell) b(r)$). Then $\bowtie$ is bilinear, and for states A, B and changes δ_A, δ_B arriving in the same step,*

$$(A + \delta_A) \bowtie (B + \delta_B) - A \bowtie B = \delta_A \bowtie B + A \bowtie \delta_B + \delta_A \bowtie \delta_B. \quad (2)$$

Proof. Bilinearity: for fixed b the map $a \mapsto a \bowtie b$ is a sum over pairs of $a(\ell)$ times a constant, hence linear, and symmetrically in b . Expand the left side of (2) by bilinearity into four terms and cancel $A \bowtie B$. $\square$

Reference algebra only: `IncrementalJoinTest: theIncrementalJoinAgreesWithRecomputingItFromScratch` (a generated property over sequences of steps) and `matchesArrivingInTheSameStepAreJoined, aRetractionOnEitherSideWithdrawsTheJoinedRow, anUpdateOnOneSideRewritesEveryRowItJoinedTo, aRowRetractedToZeroLeavesTheIndex`. `IncrementalJoin.step` takes both deltas together, because computing the left’s effect and then the right’s with the indexes updated in between counts the $\delta_A \bowtie \delta_B$ term twice.

In plain terms. When both sides of a join change at once, the change in the result has three parts: new left rows against the old right side, old left rows against the new right rows, and new against new. The third part is the one people drop, and dropping it loses exactly the matches that arrive in the same batch — a bug that hides at low traffic, because batches of one row contain no pairs, and shows up as quietly missing output when traffic rises.

Corollary 3.10 (Chains of views). *If Q_1^Δ and Q_2^Δ are incremental forms of Q_1 and Q_2 , then $(\delta, s) \mapsto Q_2^\Delta(Q_1^\Delta(\delta, s), Q_1(s))$ is an incremental form of $Q_2 \circ Q_1$; and applying a sequence of changes through an incremental form yields, at every step, the query of the accumulated input.*

Proof. $Q_2(Q_1(s + \delta)) = Q_2(Q_1(s) + Q_1^\Delta(\delta, s)) = Q_2(Q_1(s)) + Q_2^\Delta(Q_1^\Delta(\delta, s), Q_1(s))$. The second claim is induction on the number of steps using (1). $\square$

Reference algebra only: `IncrementalOracleTest: aChainOfViewsStaysCorrect, appliedDeltasAreEquivalentToRecomputingFromScratch`, with `theGeneratorProducesGenuinelyMixedInputs` guarding against a generator that only ever inserts.

3.3 What the running engine is held to

The results above are DBSP’s [7] and they are stated here because the engine’s design follows them: an update is a retraction plus an insertion, weights are added, a join is maintained by the bilinear rule, and a window correction is a retraction of the old answer followed by the new one. But it would be a misreading of the markers above to take them as evidence about the running engine, and we state why plainly.

Observation 3.11 (The reference algebra is not the executing code). The module `pravaha-algebra` — `ZSet`, `Lift`, `IncrementalJoin`, `Integrate`, `Differentiate`, `Frontier` — is imported by no production module outside itself. The engine executes its own operators over off-heap binary rows in `pravaha-runtime` (`FilterOperator`, `ProjectOperator`, `SymmetricHashJoin`, `WindowedAggregate` over `SlicedAggregateState`, `KeyedAggregate`, `TopNRanking`, `LookupJoin`), with filter and projection chains compiled to Java source and run as generated code. No test holds a runtime operator against the algebra’s oracle.

In part: this is recorded in the repository itself: ADR-013’s status reads that `pravaha-algebra`, “the DBSP oracle this ADR rests on, is imported by no module outside itself”, and a search of the tree at this commit finds no import of the package outside the module. The runtime is instead held to the algebra’s *consequences* by differential tests that compare two implementations or an implementation with hand-computed answers: `GeneratedPipelineEquivalenceTest` (`aLanePipelineGivesTheSameAnswerGeneratedAsInterpreted, the EquivalenceCatchesAGeneratedStageThatDisagrees`); `GeneratedQueryEquivalenceTest` (`aRegisteredQuery ServesTheSameViewEitherWay`); `PushdownEquivalenceTest`, `SourcePushdownEquivalenceTest` and `Partial AggregatePushdownEquivalenceTest`; `KeyedAggregatePartialEquivalenceTest`; `SharedLaneIngestProperty Test` (`queriesSharingALanesIngestAnswerExactlyAsOnLanesOfTheirOwn`); and `WindowAnswerTest`, whose tests compare windowed answers with numbers computed by hand.

Remark 3.12. Two implementations that agree are evidence that neither has drifted from the other, not that either satisfies (1). The step that would close the gap is to run the runtime’s operators and the reference algebra on the same generated changes and compare, as `IncrementalOracleTest` already does for the reference lifts. That is not built, and Section 14 lists it as the first item of future work. We regard this as the single largest distance between what the design says and what the tests show.

Two further properties of the running operators bound what the algebra can promise about them. First, `WindowedAggregate` keys its slice state by a 128-bit digest of the group columns rather than by the values (finding W8-14 in the repository’s register): two groups whose digests collide would be merged. The class comment records this as a deliberate simplification on the per-row path, with the emitted-results map already keyed by values. The algebra’s statements are about exact keys; the runtime’s are about keys up to a 128-bit collision. Second, `MIN` and `MAX` are not invertible under retraction, so they are never pre-combined at a source; the runtime keeps what it needs to recompute them. Proposition 3.6 is the justification for that choice rather than a description of it.

3.4 The served view

A registered query’s answer is a *keyed* view: a map from the declared key to one row. That is not a Z-set, and the mapping between the two is where a real defect was found and fixed, so it is worth stating exactly.

Definition 3.13 (Keyed view of a Z-set). Let $\kappa: O \rightarrow K$ extract the declared key from an output row. For a Z-set $v \in \mathbb{Z}[O]$ and a key k , let $w_v(k) = \sum_{o: \kappa(o)=k} v(o)$ be the *net weight* of k . The *keyed view* of v holds k exactly when $w_v(k) > 0$.

Proposition 3.14 (Presence is net weight). *Let a sequence of weighted changes $(o_1, w_1), (o_2, w_2), \dots$ be applied to `ServedView` and committed, and let $v = \sum_i w_i \cdot [o_i]$. After the commit, a key k is present in the view if and only if $w_v(k) > 0$. A change of weight zero changes nothing.*

Proof. `ServedView.applyWeighted` maintains, per key, the committed net weight plus the pending net weight and records a tombstone in the pending overlay exactly when the sum is ≤ 0 and the row otherwise; `commit` moves the overlay into the visible map and folds the pending weights into the committed ones, removing a key whose net is ≤ 0 . A weight of 0 returns before touching anything. By induction on the number of applied changes the stored net weight of k equals $w_v(k)$, and presence is decided by its sign. $\square$

In part: `ServedViewTest: aNegativeWeightRemovesTheKey, anUpdateIsARetractionAndAnInsertAndNeedsNo UpdatePath`; the class comment records the defect this replaced, in which a single -1 removed a key that three inserts had put there because presence was decided by the sign of the last weight. What the proposition does *not* settle is *which row* a present key holds, which is the next paragraph.

The view stores, for a present key, the values of the most recent change that left the key’s net weight positive — and that change may be a retraction. If a key is inserted as row A , then

as row B (net weight 2), and then A is retracted (net weight 1), the view holds A 's values, while the row still present in v is B . The repository records this as finding VIEWW-1, open, found by reading and not reproduced. The engine's own operators emit an update as the retraction of the old row *before* the insertion of the new, within one batch, and a batch reaches the view whole, so the case needs a source or a query that holds two distinct rows under one key at once. We therefore state the stronger property as conditional:

Proposition 3.15 (The keyed view is the Z-set, under key-functional input). *If at every commit each present key has exactly one row of positive weight in v (the output is key-functional), and within a batch a key's retraction precedes its replacement, then after each commit the view maps each present key to that row.*

Argued, not tested: from `ServedView.applyWeighted` as above; no test generates inputs that violate key functionality and checks the row. The disposition of VIEWW-1 is "a Z-set property test on the view first". The registry's refusals are the practical guard: a query whose output can repeat a key over a source that repeats rows is refused an aggregate (PRV-2042) and a keyed sink.

In plain terms. The answer is stored as "one row per key". A key is there while the pluses and minuses for it add up to more than zero, and that part is right and tested. Which row the key shows is right as long as the query never has two different rows for one key at the same time — which is what declaring a key means — and there is an open note in the project's own defect register about the case where it does.

3.5 Event time, watermarks, windows and lateness

Definition 3.16 (Watermark). For a source partition p with out-of-orderness ω , let $t_p^{\max}$ be the largest event time it has delivered; its *watermark* is $wm_p = t_p^{\max} - \omega$ (or a value the source punctuates, for a source that says so). A partition that has delivered nothing for the idle timeout is *idle*. A lane's watermark is

$$wm = \max\left(wm^{\text{prev}}, \min_{p \text{ not idle}} wm_p\right),$$

held at its previous value when every partition is idle.

Proposition 3.17 (The watermark is monotone and a quiet partition does not freeze it). *The lane watermark never decreases. A partition with no data for the idle timeout stops holding it back, and rejoins the minimum as soon as it produces again.*

Proof. Monotonicity is the outer max with the previous value; `WatermarkTracker` reports the highest watermark it has reached, and a partition whose own generator regresses is counted rather than obeyed. Idle exclusion is the restriction of the min to non-idle partitions. $\square$

Runs: `WatermarkTrackerTest: theLaneWatermarkIsTheMinimumAcrossItsPartitions, oneQuietPartition DoesNotFreezeEveryWindow, anIdlePartitionRejoinsAsSoonAsItProducesAgain, anOutOfOrderRecord CannotPullTheWatermarkBack, aGeneratorThatRegressesIsCountedRatherThanObeyed, whenEveryPartition IsIdleTheWatermarkHolds`. The reference `Frontier` refuses to regress (`FrontierTest: refusesToRegress`); a view refuses a commit whose frontier goes backwards (`ServedViewTest: aFrontierGoingBackwardsIsRefused`).

A tumbling or hopping window $[s, e)$ *fires* when the watermark passes e : its aggregate is emitted at weight $+1$. A record for a window that has fired has one of three outcomes, and the distinction between the second and third is the one that matters.

Definition 3.18 (Three outcomes of a record). Let a record with event time t belong to a window $[s, e)$, let wm be the watermark when it is processed and $\lambda \geq 0$ the stream's allowed lateness. The record is *on time* if $wm < e$; a *correction* if $e \leq wm < e + \lambda$; *too late* otherwise.

Proposition 3.19 (A correction consolidates to exactly the difference). *If a window $[s, e)$ for group g has emitted the aggregate row a and a correction for it arrives, the operator emits $-1 \cdot a$ and $+1 \cdot a'$, where a' is the aggregate including the record. If $a' = a$ the two cancel and nothing is emitted. A too-late record changes no state, produces no output, and is counted on the late output.*

Proof. `WindowedAggregate` keeps each published row, keyed by the group's values and the window's start, so it can withdraw exactly what it sent; a correction reopens the window's slices, recomputes, and emits the retraction and the new row in one batch, which consolidates in the Z-set sense ($a + 1$ and $a - 1$ of the same row cancel). A too-late record's state has been released, so it cannot be applied without contradicting an answer already sent; it is routed to the late output instead. $\square$

Runs: `LateDataTest: aLateRecordRetractsTheOldAnswerAndEmitsTheCorrectedOne, aCorrectionThatChangesNothingEmitsNothing, aRecordTooLateToCorrectAnythingGoesToTheLateOutput; StreamAllowed LatenessTest: aDeclaredAllowedLatenessReachesTheWindowedAggregate`. Allowed lateness defaults to zero, so without a declaration a window is final when it fires.

Worked example: a corrected window

A ten-second window $[10, 20)$ for user `u42` has three completed transactions totalling 90 when the watermark passes 20, and the view shows $(20, \text{u42}, 3, 90)$. The stream declares an allowed lateness of thirty seconds. At watermark 27 a transaction for `u42` stamped 15 with amount 10 arrives. The operator emits $(20, \text{u42}, 3, 90)$ at -1 and $(20, \text{u42}, 4, 100)$ at $+1$ in the same batch. A reader of the view sees the old row until the commit and the new row after it, never neither. A subscriber sees both changes in one commit, and a consumer summing weights ends with the new row. Had the transaction arrived at watermark 55, past $20 + 30$, it would have been counted on the late output and the view would still say 3 and 90.

Remark 3.20 (What event time does and does not make deterministic). The project's documentation says that loading the same rows in any order gives the same answer. That holds for the multiset of on-time records: windowed aggregates are sums over the window's records, so their final value does not depend on arrival order. It does not hold across the boundary of Definition 3.18: whether a record is a correction or too late depends on the watermark when it arrives, and the watermark depends on arrival order and, through the idle timeout, on wall-clock time. Two runs over the same rows can therefore differ in which rows were dropped as too late. The engine makes the difference visible — too-late rows are counted — but it does not remove it, and neither can any engine that closes windows on a watermark.

3.6 State that is bounded, or refused

Proposition 3.8 says that a maintained view is the integral of its changes, and an integral over an unbounded key space grows for ever. PRAVAHA therefore refuses at registration any query whose state it cannot bound: an unwindowed `GROUP BY` over a stream (PRV-2050), a stream-to-stream `LEFT JOIN` without a time bound, a join with no equality, `RIGHT` and `FULL OUTER` joins (PRV-2020). Every integrating operator is bounded by a window, a join's match window, a top- N 's per-partition ceiling, or a view's retention; and where a bound is a capacity rather than a meaning — a view's maximum key count, a join side's maximum rows — exceeding it is a refusal (PRV-4022, PRV-4001) rather than a silent eviction. A bound that changes the answer belongs in the query; a bound that protects the machine belongs in configuration and fails loudly. This is a design rule rather than a theorem, and it is enforced by the planner's refusals, which the documentation's SQL reference lists and a test plans against the engine.

4 Reading a maintained answer: commits and subscriptions

This section is H1: a client that wants to hold a copy of the view, and to keep it current, must be given a starting state and a stream of changes that meet at a seam with nothing between them and nothing on both sides.

4.1 Commits

Definition 4.1 (Commit sequence). For one view, a *commit* is the publication of every change applied since the previous one. Write $c_1, c_2, \dots$ for the commits in the order they take the view’s publish lock, Δ_k for the changes of c_k (a list of weighted rows) and V_k for the view’s committed contents after c_k , with V_0 the contents at registration or at restore.

Proposition 4.2 (A commit is atomic to readers). *A consistent read or a scan of the view returns V_k for some k ; it never returns a state containing part of Δ_{k+1} . A batch finished by a lane reaches the pending overlay whole, so a commit never contains a retraction without the insertion that accompanied it in the same batch.*

Proof. `ServedView.commit` moves the whole pending overlay into the visible map inside one monitor, and `readCommitted` and `scan` take the same monitor and read only the visible map. A lane’s batch is applied by `ViewSink` under its publish lock through `applyBatch`, which is itself synchronized on the view, so a commit, which takes the same publish lock, cannot fall between two rows of a batch. $\square$

Runs: `ServedViewTest: aConsistentReadNeverSeesHalfABatch`, `aScanSeesTheCommittedRowsAndNotThePendingOnes`, `aLatestReadSaysWhenItMayIncludeUncommittedWork` (a *latest* read may, and says so). The batch-whole property is the fix recorded as VIEW-1 in the `ViewSink` and `ServedView` comments.

Proposition 4.3 (A commit’s audience is fixed at its first batch). *A listener registered with `onCommit` receives Δ_k for exactly those commits c_k whose first batch was applied after it was registered, and receives each of them whole.*

Proof. `ViewSink.apply` sets `batchAudience` to a copy of the listener list when the commit’s first batch is applied, under the publish lock, and later batches of the same commit reuse it. At the commit the staged changes are taken as one list for that audience. A listener added mid-commit is not in the copy. $\square$

Runs: `SnapshotHandoffTest: theCommitInFlightReachesNeitherAPlainSubscriptionNorTheReadBesideIt`. The previous behaviour, recorded as STRM-11, decided the audience per row and delivered a commit’s tail to a subscriber that attached mid-commit, as though it were a whole commit.

Observation 4.4 (A plain subscription plus a read is gapful). A client that reads V_j and registers a plain listener can miss a commit: if commit c_{j+1} ’s first batch is applied after the read and before the registration, the listener is not in its audience and the read did not include it. Reversing the two steps does not help: registering first and reading second can miss the commit in flight at the registration, whose audience excluded the listener and whose rows the read may or may not see.

Runs: `SubscribeFromSnapshotTest: subscribingThenReadingLosesTheCommitInFlight`, which demonstrates the loss. This is the defect recorded as SUB-1; the documentation calls a plain subscription “gapful for a client keeping a copy”.

4.2 Subscribing from a snapshot

Assumption 4.5 (What the snapshot theorem assumes). (S1) Every batch application and every commit of the view takes the view sink’s publish lock.

(S2) The listener does not throw from `onSnapshot`, and the subscription that carries it to the client does not overflow its buffer. (An overflowing snapshot subscription is ended with PRV-6105 rather than written past the gap.)

(S3) Deliveries of successive commits reach the listener in commit order. Delivery happens outside the lock, on the thread that ran the commit.

Theorem 4.6 (Snapshot, then every commit exactly once). *Under (S1)–(S2), a listener attached with `onCommitFromSnapshot` at a moment when c_m is the last completed commit receives first `onSnapshot( $V_c, f_c$ )` for some $c \geq m$, and then `onCommit( $\Delta_k$ )` for every $k > c$ exactly once and for no $k \leq c$. With (S3) the deliveries arrive in increasing k . Consequently, a consumer that starts from V_c and applies each delivered change by weight holds V_k after the delivery of c_k , for every $k > c$.*

Proof. The attach runs in one critical section of the publish lock, which by (S1) orders it against every batch and every commit. Two cases.

No commit in flight: `batchAudience` is null and the view has no pending changes, so every change applied so far is committed and the committed state is V_m . The listener’s snapshot is captured as V_m with its frontier and the listener is added to the listener list inside the same critical section. The next batch to be applied is the first of c_{m+1} and, by Proposition 4.3, copies a list that contains the listener; so the listener is in the audience of every c_k with $k > m$ and of none with $k \leq m$. Take $c = m$.

A commit in flight: c_{m+1} has applied at least one batch, its audience was fixed without the listener, and its rows are not yet committed. The listener is placed on the joiners list instead. c_{m+1} ’s own critical section, after `view.commit`, calls `promoteJoiners`, which captures V_{m+1} — now containing the rows that were in flight — as the snapshot and adds the listener to the list, still inside the lock. As before the listener is in the audience of every later commit and, since c_{m+1} ’s audience was fixed before, not of c_{m+1} . Take $c = m + 1$.

Exactly once: each commit’s changes are delivered once to each member of its audience, and the listener is a member exactly for $k > c$. The snapshot is delivered once, before any `onCommit`: the `Handoff` wrapper’s `onCommit` calls `handOver` first under its own monitor, and `handOver` clears the snapshot after delivering it. By (S2) nothing is dropped on the way to the client. The final sentence is induction on k using $V_k = V_{k-1} \oplus \Delta_k$, where $\oplus$ is the keyed application of Section 3.4; for a consumer that sums weights rather than overwriting by key, order is irrelevant by commutativity of $+$, so (S3) is needed only by a consumer that overwrites. $\square$

Runs: `SnapshotHandoffTest: aSnapshotSubscriptionAttachedMidCommitIsHandedTheViewThatCommit Produced, withNoCommitInFlightTheSnapshotIsTakenAtOnceAndTheNextCommitIsDeliveredWhole, anEmpty ViewStillSendsASnapshotAndARowPresentTwiceArrivesWithItsWeight, everySubscriberAttachingDuring ConcurrentCommitsEndsWithTheView; SubscribeFromSnapshotTest: aSnapshotSubscriptionAttachedMid CommitStartsWithTheRowsInFlight, everySnapshotSubscriberOfABusyAggregateEndsHoldingTheView, a ListenerThatThrowsOnItsSnapshotIsDetachedAndSaysWhy; EmbeddedSnapshotSubscriptionTest: rows AppliedButNotYetCommittedAreInTheSnapshot; and for (S2), SnapshotSubscriberBehindTest: aSnapshot SubscriberTooFarBehindIsEndedWithPrv6105AndNeverWrittenPastTheGap.` The tests assert the end state — snapshot plus deltas equals the view — under concurrent commits; (S3) as an ordering property is argued from the call sites and not asserted separately.

In plain terms. “Give me the table, then every change after it” is one step, not two, and it is done under the same lock that publishes changes. If nothing is half-published, you get the table now and are first in line for the next change. If something is half-published, you wait for it to

finish, and then you get the table *including* it and are first in line for the one after. Either way there is exactly one place where the table stops and the changes start, and nothing falls into the crack or lands on both sides of it.

Worked example: a dashboard that reconnects

A risk dashboard keeps a local copy of `txn_volume` to draw a heat map. It subscribes from a snapshot and receives 40,000 rows at frontier F , then a commit every second. The engine restarts for an upgrade. The Java SDK’s reconnecting subscription reopens the stream with back-off and, because the original was a snapshot subscription, the first batch after reopening is a *fresh* snapshot at the restored view’s frontier rather than a stream of deltas the client would have to splice onto a copy it can no longer trust. The client replaces its copy and continues. What it never does is apply a delta across a restart it cannot see (`JavaSdkReconnectTest`).

Remark 4.7 (What is not offered: a read across views at one point). Each view commits on its own schedule. A client reading two views gets each at some commit of its own, and the **AtLeast** consistency mode lets it wait for each to reach a frontier; it does not get both at the *same* frontier. The reference **Frontier** class describes cross-view consistency as the purpose of frontiers, but no read in the engine returns several views at one frontier, and an **AsOf** read is refused (`ServedViewTest.anAsOfReadIsRefusedRatherThanAnsweredWithThePresent`) because a view holds the present and the past lives in checkpoints.

5 Exactly once: a checkpoint is one cut

This section is H2. It is the oldest of the four problems and the one with the most careful prior treatment [12, 13, 15]; we state it for this engine because the guarantee is only as good as the assumptions it is proved under, and those are specific to how a single node reads its sources and feeds its lanes.

5.1 The setting

Definition 5.1 (Log, positions, reader). A source partition p is a sequence of records $r_1^p, r_2^p, \dots$ with *end positions* $\pi_1^p \prec \pi_2^p \prec \dots$ in a set of positions totally ordered by $\prec$, with a least element $\perp$ (the beginning). The position after consuming r_i^p is π_i^p ; a position may also fall strictly between two end positions (a gap, such as a Kafka transaction marker). For a position a , write $\text{rec}_p(a, b] = (r_i^p : a \prec \pi_i^p \preceq b)$ for the records after a up to and including b , in order.

Assumption 5.2 (What exactly-once rests on). (A1) *Replayable positions*. A reader created at a position a that it or another reader of the same partition handed out delivers $\text{rec}_p(a, \infty)$, in order, and its reported position after delivering r_i^p is π_i^p (or a later gap position before π_{i+1}^p). The source retains the records after any position a durable checkpoint recorded.

- (A2) *Deterministic operators*. The state and output of a lane after processing a sequence of input rows is a function of that sequence.
- (A3) *One lane, no exchange*. A registered query’s pipeline runs on one lane; no row is in flight between two lanes of one query.
- (A4) *FIFO inbox*. A lane processes the rows of its inbox in the order they were written.
- (A5) *Durable checkpoint store*. A checkpoint is either durably stored whole or not at all, and a restore reads the newest one stored whole.

In plain terms. Exactly-once is not something an engine can give on its own. It needs the source to be able to replay from a bookmark (a table scan that reports where its pass began cannot), the operators to give the same answer for the same input, and the checkpoint to be written all or nothing. The theorem below says what the engine adds on top of those; the assumptions are the parts it borrows.

5.2 The aligned cut

`QueryExecution.checkpoint` runs in two phases. In the first, every source pump of the query is *frozen*: `IngestPump.pumpOnce` holds a lock for the length of a poll, and the checkpoint takes that lock through `freezeIngest`, so each pump is held between two rows it has fully written into the inbox. Inside the freeze the checkpoint reads every source's offset, and hands the lane a *control task* — the marker — at the position its producers have reached. Nothing is waited for inside the freeze. In the second phase the sources run again and the checkpoint waits for the lane to run the task.

Lemma 5.3 (The freeze reads positions and places the marker at one point). *Let o_p be the offset read for partition p during the freeze and let M be the marker's position in the lane's input. Every row the pumps wrote before the freeze is ahead of M , every row they write after is behind it, and the rows ahead of M from partition p are exactly $\text{rec}_p(o_p^0, o_p]$, where o_p^0 is the position the query started or last restored from.*

Proof. While frozen, no pump is inside a poll, so no row is part-way between a reader and the inbox, and each pump's reported offset is the end position of the last record it delivered (or a gap position after it), by (A1). The marker is submitted at the producer frontier while the pumps are held, so by (A4) every row written before is ahead of it; rows written after the thaw are behind it. The rows a pump wrote since o_p^0 are, by (A1), $\text{rec}_p(o_p^0, o_p]$. $\square$

Lemma 5.4 (A marker is honoured exactly). *The lane runs the control task after processing every row ahead of M and before any row behind it, even if M falls in the middle of what would otherwise have been one batch.*

Proof. A lane sizes each batch against the head of its control queue: a batch is cut at a queued marker and at the producer frontier, so the task runs at a batch boundary equal to M . $\square$

Runs: `ControlTaskBarrierTest` in `pravaha-runtime`: `aTaskSeesTheStreamAtItsMarkerAndNotABatchBeyondIt`, `aCutQueuedBehindALevelStillBindsTheBatch`, `everyInputOfAJoinIsCutAtItsOwnMarker`. ADR-008 records that before the fix a lane finished the batch it was in, putting rows into the snapshot that the recorded offset said would be replayed.

Theorem 5.5 (A checkpoint is one consistent cut; state is exactly once). *Under Assumption 5.2, let a checkpoint record offsets $(o_p)_p$, the lane's operator state σ and the view's committed contents V , all taken as above. Then σ and V are the state and the view after processing exactly the rows $\bigcup_p \text{rec}_p(o_p^0, o_p]$, in the order the lane received them. A restore from this checkpoint followed by reading each partition from o_p processes every input record exactly once in total: records up to o_p are reflected in the restored state and are not read again, and records after o_p are read once and were not reflected.*

Proof. By Lemma 5.3 the rows ahead of the marker are exactly the rows up to the recorded offsets; by Lemma 5.4 the snapshot is taken after all of them and none of the others; by (A3) there is no second lane holding part of the query's input and no row in flight between lanes; and the view's contents are cut in the same control task, on the lane, at the same marker (`cuttingOutputWith`), so V describes the same point. On restore, by (A5) the whole checkpoint is read; by (A1) a reader at o_p delivers exactly $\text{rec}_p(o_p, \infty)$. The union of what the state reflects and what the readers deliver is each partition's whole log, with no record in both. $\square$

Runs: `AlignedCheckpointBarrierTest` in `pravaha-it`: `aCheckpointTakenMidStreamAccountsForExactlyTheRowsItsOffsetExcludes` and `aMultiLaneCheckpointRecordsEverySourceAndCutsThemAtOnePoint`, both taking checkpoints with the sources still running; `CheckpointRecoveryTest`: `aRunInterruptedAndRecoveredProducesTheSameAnswersAsAnUninterruptedOne`; `AggregateRestartTest`: `aWindowedAggregateRestartedMidWindowFinishesItWithEveryRow`, `aStreamJoinRestartedMatchesARowItHeldAcrossTheRestart`, `aGlobalAggregateCheckpointedTwiceAcrossTwoRestartsStillHoldsOneRow`. The assumption's necessity is itself shown: `StateRestoreTest`'s `state057_restoringWithoutRewindingTheSourcesDoubleCountsEverythingSinceTheCheckpoint`, and a checkpoint from a plan with a different number of stateful operators is refused rather than restored (`state058_aSnapshotFromAPlanWithADifferentNumberOfStatefulOperatorsIsRefused`).

In plain terms. A checkpoint stops every reader between two rows, writes down each reader's bookmark, and drops a marker into the query's input right there. The query saves its work when it reaches the marker — not a batch later. So the saved work covers exactly the rows before the bookmarks. After a crash, the saved work is loaded and every reader starts at its bookmark: every row is counted once, either in the saved work or by being read again, never both and never neither.

Remark 5.6 (What “exactly once” does and does not mean here). Theorem 5.5 is about *effect*: each record's contribution is in the state exactly once. It does not say that the resumed run is bit-identical to an uninterrupted one. After a restore, rows from different partitions may reach the lane in a different interleaving than they would have, and the watermark (Remark 3.20) may advance at different moments. For operators whose output is a function of the multiset of on-time records — the windowed aggregates — the answers agree, which is what **CheckpointRecoveryTest** checks; for anything sensitive to the interleaving the resumed run is *an* execution of the same input, not necessarily the same one.

Remark 5.7 (The two assumptions the engine enforces by refusal). (A3) is enforced rather than assumed. A multi-lane checkpoint of an execution whose rows cross the exchange between lanes would not be a cut — a row one lane has sent and another has not drained belongs to neither snapshot and to no offset — and forwarding the marker along each ring is not built. `QueryExecution.refuseWhileRowsCrossTheExchange` refuses such a checkpoint, and a checkpoint that cuts a view's output is refused for an execution on more than one lane. No pipeline the engine compiles for a registered query sends on the exchange, so the refusal is unreachable today; it exists so that the first pipeline that does finds an error rather than a wrong answer. (A1) is enforced at declaration: a source that declares exactly-once without replayable offsets is refused (`CapabilitiesTest`: `exactlyOnceWithoutReplayableOffsetsIsRefusedAtDeclaration`), and Kafka refuses to resume from an offset its retention has deleted rather than skip to the next one (`KafkaSourceBrokerTest`: `recordsRetentionDeletedBeforeTheCheckpointReadThemAreRefusedNotSkipped`).

Argued, not tested: the exchange refusal is read from `QueryExecution`; no test drives a pipeline that sends on the exchange, because none exists.

5.3 Output: two-phase sinks

State that is exactly once is not output that is exactly once. A sink written on every commit is written again, after a restore, for every commit between the checkpoint and the crash. The standard answer is to tie the sink's transactions to checkpoints [15, 22]; the protocol PRAVAHA's sink interface (`StreamSinkPlugin`) states is:

1. `beginTransaction(n)`, then writes.
2. At a checkpoint's cut, on the lane, `prepare(id)`: make everything written since the begin durable and *not visible*, and return a handle naming it. The handle is stored in the checkpoint; the next transaction begins at once.

3. Once that checkpoint is durable, `commit(handle)`.
4. After a restart: restore the newest checkpoint, call `commit` for every handle it recorded, then `abortAfter(id)` for everything else.

Assumption 5.8 (What the sink must supply). (K1) `prepare` makes the transaction durable and invisible, and its handle names it across processes.

(K2) `commit` is idempotent: committing a handle already committed changes nothing.

(K3) `abortAfter(id)` abandons every uncommitted transaction begun after checkpoint `id`, or they die with the connection that opened them.

Theorem 5.9 (Exactly-once output to a transactional sink). *Under Assumptions 5.2 and 5.8, every change the view commits is made visible in the sink exactly once across any sequence of crashes and restores.*

Proof sketch. Partition the view’s changes by the checkpoint interval in which they were committed. By Theorem 5.5 and the output cut, the transaction prepared at checkpoint j contains exactly the changes between the cuts of $j - 1$ and j . Consider a crash at any point. *Before checkpoint j is durable:* the newest durable checkpoint is $j - 1$; its handle is committed (again, harmlessly, by (K2)); the transaction prepared at j , if any, and the open one are abandoned by `abortAfter` (K3); and the replay from $j - 1$ ’s offsets regenerates their changes, which go into new transactions. *After j is durable and before its commit is sent:* the restore commits j ’s handle, which by (K1) still names the prepared data, and the replay from j ’s offsets regenerates only later changes. *After the commit:* the restore commits it again, a no-op by (K2). In every case each interval’s changes become visible in exactly one committed transaction. $\square$

Runs: TransactionalSinkDeliveryTest in pravaha-registry: nothingIsVisibleUntilTheCheckpointRecordingItIsDurableAndThenAllOfItIs, aCrashBetweenPrepareAndCommitLosesNothingAndRepeatsNothing, writesAfterTheRestoredCheckpointAreAbortedAndWrittenAgainOnce, aRowAppliedButNotYetCommittedWhenTheCheckpointIsTakenIsInsideIt, aCheckpointThatFailsAfterItsCutLeavesItsTransactionToTheNextOne, theGuaranteeIsStatedPerKindOfSink; against real systems, KafkaSinkBrokerTest: aCrashBetweenPrepareAndCommitThenARestoreWritesEachChangeExactlyOnce, and aCrashAfterTheCheckpointBeforeTheCommitLosesNothingAndRepeatsNothing in KafkaSinkRegistrationTest and DeltaSinkRegistrationTest.

How a sink supplies (K1) varies, and the variation is a cost. Kafka has no prepare that a restarted producer could commit, and a Delta commit is visible the instant its log entry lands, so `kafka-sink`, `delta-sink` and (by default) `jdbc-sink stage` each checkpoint’s changes — in a staging topic, a staging directory, a staging table — and apply them in one transaction once the checkpoint is durable. Every change is written twice and the output trails the view by up to a checkpoint interval. `jdbc-sink` over PostgreSQL can instead use the database’s own `PREPARE TRANSACTION` (`commit.mode: prepared`), writing each change once at the cost of row locks held for an interval. And `kafka-sink` is exactly once only to a consumer reading `read_committed`.

Proposition 5.10 (The end-to-end guarantee is the weakest link). *Order the guarantees $AT-MOST-ONCE < AT-LEAST-ONCE < EXACTLY-ONCE$. The guarantee a registration states is the minimum of its source’s, the engine’s and its sink’s. A source that repeats rows cannot offer exactly once; an idempotent upsert sink raises at-least-once delivery to an exactly-once effect; a plain append sink is at least once.*

Runs: DeliveryGuarantee.weakest; CapabilitiesTest: aSourceThatRepeatsRowsCannotOfferExactlyOnce, aTransactionalSinkCanOfferExactlyOnce, anIdempotentSinkAlsoReachesExactlyOnceWithoutTransactions; SinkGuaranteeListingTest: aTransactionalSinkOnACheckpointedNodeIsExactlyOnce, withoutCheckpointsNothingIsListedExactlyOnce.

In plain terms. The promise a query can make is the weakest promise in its chain. A source that cannot rewind to a bookmark — an Aerospike scan, for instance — makes everything downstream at-least-once, whatever the sink can do, and the registration says so rather than letting a transactional sink’s name imply more.

6 Sharing a reader at an exact seam

This section is H3. A thousand queries over one Kafka topic should read it once, not a thousand times. Sharing a reader among queries that all start together is easy. The difficulty is that queries do not start together: a query is registered while others are running; a query is paused and resumed; after a restart each query restores from its own checkpoint, taken at its own moment, so the members of a group resume at different positions. Some are behind the shared reader and some are ahead of it.

6.1 Why the earlier sharing could not be exact

The engine’s first form of reader sharing (recorded as SRC-3) attached a joining query to the fan-out at once and gave it a private catch-up reader for the history it had missed. The catch-up read to the end of the source, not to where the shared reader stood, so the overlap arrived twice and history and live records arrived interleaved. For a source that promises at least once and no order — an Aerospike or Cassandra scan — that is within the promise, and SRC-3 is still how those sources are shared. For a source that promises exactly once or order it is not, so until ADR-054 every such source kept a reader per query.

The obvious repair — stop the catch-up after reading “the difference in offsets” records — is wrong for Kafka, whose offsets have gaps: a transaction’s commit marker occupies an offset and is not a record, so the number of records between two offsets is not their difference, and a count-bounded catch-up overshoots the seam.

6.2 Two declarations

Definition 6.1 (Ordered positions). A source *declares ordered positions* when it supplies `compare(a, b)` (`OrderedPositions`) over positions its readers handed out for the same partition, negative when a comes before b , zero when they name the same position, and refuses a token it cannot read. We write $a \prec b$.

Definition 6.2 (Bounded read). A reader supports a *bounded read* when it implements `pollBefore(sink, max, b)` (`BoundedPartitionReader`) with the contract: deliver, in order, only records whose end position is $\preceq b$; once no such record remains, report b as the position; a bound at or before the current position reads nothing; a record after b is not consumed.

A source declares the first by returning a non-null `orderedPositions()`, and whether it may can depend on configuration: a followed file restarts its line count when the file is replaced, so its positions are ordered only while it is not followed (`FilesystemPluginTest: aFollowedFileDoesNotClaimOrderedPositions`). The filesystem reader implements the bounded read from the line count; the Kafka reader compares each record’s offset with the bound, and a control record spanning the bound moves the position to exactly the bound.

Assumption 6.3 (What the seam theorem assumes). (B1) `compare` is a total order on handed-out positions of a partition that agrees with the order of Definition 5.1.

(B2) Every reader of the partition keeps Definition 6.2 and (A1).

- (B3) All polls of the shared reader and of every catch-up, all comparisons, joins, leaves, pauses and resumes of one partition's group run under one lock on one feed thread.
- (B4) A shared poll happens only when every attached, unpaused member has room in its inbox; when one does not, the poll is skipped for all of them rather than performed with that member's copy dropped.

6.3 Three states

Definition 6.4 (Member state). Let σ be the shared reader's position. A member m of the group joined at position a_m and is in one of three states: *attached*, receiving every record the shared reader reads; *catching up*, with a private reader at position $c_m \prec \sigma$ read only with `pollBefore`(σ); or *waiting*, at a position $w_m \succ \sigma$, receiving nothing. A paused member is in none of them and receives nothing. Define the member's position q_m as σ , c_m or w_m respectively.

The protocol, as `SharedPartitionFeed` implements it, is:

Join at a . If some member is live: compare a with σ . If $a \prec \sigma$ the member is catching up with a reader created at a ; if $a \succ \sigma$ it is waiting at a ; if equal it is attached. If no member is live, the shared reader is simply moved to a and the member attached: nobody is being fed from σ , so it is not a seam (LANE-6).

Catch-up round. For each catching-up member, if $c_m = \sigma$ attach it; if $c_m \succ \sigma$ (possible only if the shared reader was moved back) make it waiting at c_m ; otherwise poll its reader with `pollBefore`(σ) into its own route. Catch-ups are polled up to eight times for each shared poll, so a catch-up closes on a seam that keeps moving.

Shared poll. Let b be the earliest waiting position, if any. If no member is attached and some member is waiting, move the shared reader to b and attach every member waiting at b . Otherwise poll the shared reader — with `pollBefore`(b) if a member is waiting, plainly if not — into every attached member, then attach every waiting member whose position is $\preceq \sigma$.

Grace. A catch-up that makes no progress toward its seam for thirty seconds stops the group with a named failure: its source's positions do not behave as declared, and reading on would pass the seam.

Lemma 6.5 (A waiting member is attached exactly at its position). *Whenever a waiting member m is attached, $w_m = \sigma$.*

Proof. While any member waits, the shared reader is polled only with `pollBefore`(b) where b is the earliest waiting position, so by (B2) $\sigma \preceq b \preceq w_m$ after every poll, and $\sigma = b$ once nothing before b remains. A waiting member is attached only when $w_m \preceq \sigma$, which with $\sigma \preceq w_m$ gives equality. The other route to attachment, moving the shared reader to b , sets $\sigma = b$ and attaches exactly the members waiting at b . A member becomes waiting only at a position $\succ \sigma$ (on join, or from a catch-up that overshot a reader moved back), so the invariant $\sigma \preceq w_m$ holds from the moment it waits. $\square$

Theorem 6.6 (Each member receives each record exactly once, in order). *Under Assumption 6.3, at every moment, the records a member m has received since it joined at a_m are exactly $\text{rec}(a_m, q_m]$, in order, where q_m is its current position; and the offset any checkpoint of m records is q_m .*

Proof. By induction over the steps of the protocol, each of which runs atomically by (B3). At the join, nothing has been received and $q_m = a_m$ in all three cases, since $c_m = a_m$ or $w_m = a_m$ or $\sigma = a_m$. We show every step preserves the invariant for every member.

Catch-up poll. m receives the records `pollBefore`(σ) delivers from c_m , which by (B2) are $\text{rec}(c_m, c'_m]$ for its new position $c'_m \preceq \sigma$; appending them to $\text{rec}(a_m, c_m]$ gives $\text{rec}(a_m, c'_m]$. No other member receives them: they go to m 's own route.

Attaching a catching-up member happens only at $c_m = \sigma$, so q_m does not change.

Shared poll. The records read are $\text{rec}(\sigma, \sigma']$ for the new position σ' (by (B2) when bounded, by (A1) otherwise). Each attached member had received $\text{rec}(a_m, \sigma]$ and now receives the continuation, so the invariant holds at σ' . By (B4) the poll happens only if every attached, unpaused member receives it, so no attached member's sequence has a hole. Catching-up and waiting members receive nothing and their positions do not change.

Attaching a waiting member happens, by Lemma 6.5, only when $w_m = \sigma$.

Moving the shared reader happens only when no member is attached, so no member's received sequence is defined by σ at that moment.

Pause and resume. A member paused off the shared reader stops at an exact record and records it as its position; resuming joins it again at that position, which reduces to the join case.

Order and exactly-once follow from the invariant, since $\text{rec}(a_m, q_m]$ is a sequence without repetition in log order. The checkpoint claim holds because a checkpoint of m reads its offset through the feed's lock, where it is q_m by definition of the state. $\square$

Runs: `ExactSharingTest` in `pravaha-bindings`, over `OrderedLogPlugin` (an ordered, exactly-once test source whose positions are record counts and which counts every record any reader hands out): `aLateJoinerCatchesUpToTheSeamAndThenSharesWithNothingTwice`, `aPausedQueryResumesThroughAnExactCatchUpWhileTheOtherKeepsReading`, `aQueryRestoredAheadOfTheSharedReaderWaitsForItAndIsHandedNothingTwice`, `theSameSourceWithoutDeclaringAnOrderKeepsAReaderPerQuery`; the measure is rows in, not the view, because a keyed view absorbs a duplicate that ingest counted. The filesystem bounded read: `FilesystemPluginTest: aBoundedReadStopsOnTheLineNumberEvenWithBlankLinesInTheWay`.

In part: Kafka's `pollBefore` is not exercised directly by any test in the Kafka plugin; its gap handling is tested for ordinary reads (`KafkaSourcePluginTest: offsetsWithNoRecordAreSteppedOverOnlyOnceEverythingBeforeThemIsHandedOver`), and the protocol is tested over the test source above rather than over a broker. A broker-backed test of a late joiner over a topic with transaction markers across the seam is the missing piece.

In plain terms. One reader walks down the log, and every query standing where it stands gets each record from it. A query that shows up *behind* gets a private reader that is allowed to read only up to where the shared one is — not “to the end”, and not “this many records”, but “stop exactly at this bookmark”. When the two bookmarks are equal the query switches over. A query that shows up *ahead* waits; the shared reader is told to stop exactly at the waiting query's bookmark, and the query joins when it gets there. Every query gets every record once, in order, whichever of the three situations it arrived in.

Worked example: a restart with three queries at three positions

Three queries read partition 0 of topic `payments`. Before a restart their newest checkpoints recorded offsets 1,000, 1,400 and 1,250. On restart the first to rejoin, q_1 , has nobody else live, so the shared reader is created at 1,000. q_2 joins at 1,400, ahead: it waits. q_3 joins at 1,250, also ahead: it waits. The shared reader is polled with `pollBefore(1,250)` and hands q_1 the records up to exactly 1,250 — including, if offset 1,250 is a commit marker, a position of exactly 1,250 without a record — and q_3 attaches there. Polled next with `pollBefore(1,400)`, it lands on 1,400 and q_2 attaches. From then on one reader feeds all three. None of them was handed a record its checkpoint already reflected, and none missed one.

The consequence the engine reports is the one the design wanted: N queries over an ordered source read it once, and a member joining, pausing, resuming or restoring costs a private read

of only the gap between its position and the shared one. A slow catch-up does not stall the group: attached members keep reading while it catches up. What *does* stall the group is a full inbox (B4), and that is a choice: the alternative is to drop the slow query’s copy, which turns backpressure into silent loss for whichever query was slowest. Change-data-capture sources are deliberately excluded. A `postgres-cdc` slot is confirmed at each checkpoint (`PartitionReader.checkpointed`), and a shared slot could be confirmed only up to the slowest member’s durable position, which needs its own design. Delta and JDBC are excluded until their positions are shown to be totally ordered.

7 Lossless replacement: a position both versions have consumed

This section is H4. A query’s SQL is changed — a filter is corrected, a column is added, a window widened — and the answer must not be taken away while the new version rebuilds the state it needs, must not be half one version’s and half the other’s, and must be recoverable if the new version is wrong.

7.1 The splice

The new version, the *candidate*, is registered beside the running version, which keeps serving. For each partition of each stream the running version reads, let S be the position it has reached when the replacement starts. The candidate reads that partition through an `OffsetSplicedReader`: a history reader from the beginning (or from the candidate’s own checkpoint, after a restart), polled *one record at a time* until its position is exactly S , and then a live reader created at S .

Lemma 7.1 (The splice delivers the log once). *Under (A1), if the history reader starts at $h_0 \preceq S$ and S is a position a reader of the partition handed out, the spliced reader delivers $\text{rec}(h_0, S]$ from the history reader followed by $\text{rec}(S, \infty)$ from the live reader, which is $\text{rec}(h_0, \infty)$ with no record twice. If the history runs out without its position ever equalling S , the splice fails with `PRV-4013` after a grace period and delivers nothing past its last position.*

Proof. Polling one record at a time, the history reader’s position after each poll is the end position of the record it delivered (or a gap after it). Since S is a handed-out position, some prefix of polls lands exactly on S , and at that moment the delivered records are $\text{rec}(h_0, S]$; no poll can straddle S , because each delivers at most one record. The live reader created at S then delivers $\text{rec}(S, \infty)$ by (A1). If the position never equals S the source’s tokens do not name records as assumed, and continuing would deliver the overlap twice, so the reader stops. $\square$

Runs: `OffsetSplicedReaderTest` in `pravaha-backfill`: `historyAndTheLiveStreamMeetAtTheSeamWithNothingLostOrRepeated`, `aSplicedReadEqualsReadingTheWholeLogWithOneReader`, `aSeamThatNeverArrivesStopsTheBackfillRatherThanReadingPastIt`, `aSeamOnARejectedRecordWithRowsAfterItStopsThereAndLosesNothing`, `aPositionTakenDuringTheHistoryCarriesTheSeamAndResumesInTheSamePhase`, `aBackfillWhoseSeamIsWhereItStartsHasNoHistoryToRead`, `aStreamWithNoSeamIsOneReaderThatBecomesLiveWhenTheHistoryRunsOut`.

Where ADR-015 and the design had described a table snapshot spliced onto a change feed with a deduplication window keyed by the store’s version (`SplicedReader`), no source plugin here exposes a snapshot read separately from its change feed; and `SplicedReader` keeps the newest row per key, which double-retracts on a weighted changelog. It stays in the tree, tested and unwired. The seam every plugin here can offer is an offset into an ordered, replayable log, so that is the seam used.

7.2 The cutover

Assumption 7.2 (What the cutover theorem assumes). (R1) Every stream the two versions read has replayable, ordered positions (A1 and B1); a stream whose positions do not order its records — a table scan — or that cannot be replayed is refused before anything starts (PRV-4018).

(R2) Pausing a version’s feed takes effect between polls; its positions are read with its sources frozen, as in Lemma 5.3; and “settled” means two successive readings of its positions are equal, after which its lanes are drained and its view committed.

(R3) The name answers to one computation; a computation shared by several names cannot be replaced (PRV-8003), because a replacement moves one name.

(R4) Both versions’ operators are deterministic (A2).

(R5) The replacement backfills (`backfill = 'history'`, the default). With `backfill = 'none'` the new version starts where the running one is, with empty state, and says so; clause (ii) below then does not hold, by design (`QueryReplacementTest: backfill NoneStartsWhereTheRunningVersionIsAndSaysSoRatherThanPretending`).

The cutover (`QueryReplacements.align`) pauses both feeds, settles both, and compares their positions *token for token*, per partition, for every stream both read. If they are equal the name is swapped in the view catalogue. If not, both feeds are resumed briefly, paused again, and compared again; a cutover that cannot find equal positions within its budget is refused with PRV-4014, and a seam that would go backwards with PRV-4015. Streams only the candidate reads have no seam; the candidate’s own history on them must be exhausted before a cutover is attempted.

Theorem 7.3 (Lossless cutover). *Under Assumptions 5.2 and 7.2, when the name is swapped from version A to version B:*

- (i) *both have consumed exactly the same prefix P of every stream they both read;*
- (ii) *B’s view is the view a registration of B from scratch would hold after consuming P (and its own history of any stream only it reads);*
- (iii) *every read of the name returns A’s view or B’s view, never a mixture: the answer to the old question up to the seam and the answer to the new question after it, with no input record in neither and none in both;*
- (iv) *every subscriber to the name receives every commit A ever made — the last one taken while it was paused at the seam — and is then ended with PRV-4019 rather than handed B’s changes;*
- (v) *a sink bound to the name is carried to B at a checkpoint boundary and sent only the difference between what it holds and B’s view.*

Proof sketch. (i) is what the alignment checks, under (R2): with the feed paused and the sources frozen, a version’s positions are exactly what it has consumed, and two equal readings mean no poll was in flight. (ii) follows from Lemma 7.1: the candidate’s input on each shared stream is $\text{rec}(\perp, S]$ followed by $\text{rec}(S, P]$, which is the prefix P read once, and by (A2) its state and view are the ones a from-scratch run over the same input computes. (iii): a read resolves the name through the view catalogue at the moment of the read and reads one view; both views are committed at the settled point (R2), so neither is behind the other, and by (i) the input boundary is the same record for both. (iv): A’s last commit happens during settling, before the

swap; after the swap A 's subscribers are ended with a named code, so none receives a change of B 's and none misses one of A 's. (v): A is checkpointed with its feed stopped, committing everything its sink was written; B 's checkpoint identifiers continue above A 's so transaction labels only increase; what the sink holds is recorded on B as a carried section, and the delivery that attaches is owed the difference between that and B 's view, written as one batch. The journal's cutover record is written between those two steps, so a crash is recoverable in either direction. $\square$

Runs: `QueryReplacementTest` in `pravaha-registry`: `afterTheBackfillTheSpliceAndTheCutoverTheViewIsTheOneARegistrationFromScratchWouldHold`, `aCutoverBeforeTheCandidateHasCaughtUpIsRefusedRatherThanLeavingAGap`, `everySubscriberIsToldTheViewWasReplacedRatherThanHandedTheOtherQueryChanges`, `anAutomaticCutoverHappensAsSoonAsTheCandidateHasCaughtUp`, `aBackfillWhoseFeedStopsFailsTheReplacementAndReleasesTheCandidate`; `BackfillSpliceEquivalenceTest` in `pravaha-bindings`: `theNewVersionHoldsTheAnswerItWouldHaveHadIfItHadAlwaysBeenRunning`, `aSourceThatCannotBeReplayedIsRefusedByNameBeforeAnythingStarts`; `ReplacementSinkHandoverTest`: `theSinkMovesWithTheNameAtACheckpointBoundaryAndIsSentOnlyTheDifference`. Clause (iii)'s "never a mixture" is argued from the catalogue lookup; no test races reads against the swap.

In plain terms. The new version replays the history up to exactly where the old version is, then reads live beside it. To swap, both are paused and their bookmarks compared, source by source, partition by partition. Only when every bookmark is equal does the name move. At that moment both have read exactly the same records, so the reader who asks just before the swap gets the old question's answer to that input and the reader just after gets the new question's answer to the same input. Subscribers are told the view was replaced instead of being quietly switched to a different query's changes, because a client adding up changes would otherwise end with a total that is half one query and half the other.

7.3 The rollback window

Proposition 7.4 (Rollback is the same alignment reversed). *For the rollback retention period after a cutover (an hour by default, or until somebody confirms), the replaced version A keeps running, fed and committing. A rollback within that window aligns A and B exactly as a cutover does, with the roles exchanged, and has the guarantees of Theorem 7.3. After the window closes, A is released and a rollback is refused: there is nothing to roll back to.*

Proof. While retained, A reads the same live streams as B from the same seam onward, so it is at a position comparable with B 's and the alignment of Theorem 7.3 applies unchanged. Once released, it has no feed and no state, and a rollback would be a second backfill, which the operation does not attempt. $\square$

Runs: `QueryReplacementTest`: `aRollbackPutsTheVersionItReplacedBackExactly`, `theRollbackWindowClosesOnItsOwnAndReleasesTheReplacedVersion`, `afterTheReplacementIsFinishedThereIsNothingToRollBackTo`; `ReplacementSinkHandoverTest`: `aRollbackTakesTheSinkBackToTheVersionThatWasThereBefore`.

The window costs a retained computation — the replaced version's lanes, state and reads — for its length. That is the price of a rollback being one swap rather than a second backfill, and it is visible: the console shows the window as a time and says when it has closed.

Proposition 7.5 (A replacement survives a restart). *The registry journal records a pending replacement (P), the name's change of version (C , one record for a cutover or a rollback), and the end of one that never took the name (E). A restart brings the name back as the version the journal last recorded as serving it, restarts a pending candidate from its own checkpoints, and a version that took the name keeps the checkpoint directory it backfilled into.*

Runs: `ReplacementRestartTest`: `aReplacementStillBackfillingWhenTheNodeRestartsIsStartedAgainFromTheJournal`, `afterACutoverTheNameComesBackAsTheNewVersionWithTheStateItBackfilled`.

The directory travels in the registration rather than the files being moved, because a move leaves a window in which a crash has the name pointing at a directory holding the other version's state — restoring one version's operator state into another's plan is the kind of wrong nothing reports. (A checkpoint from a plan with a different number of stateful operators is in any case refused, as Theorem 5.5's evidence notes.)

Worked example: correcting a filter without taking the answer away

`txn_volume` has counted PENDING as well as COMPLETED transactions for a week. The fix is `CREATE OR REPLACE CONTINUOUS QUERY txn_volume ... WITH (cutover = 'manual')`. The candidate replays the topic from the beginning at a throttled rate the operator may lower but not raise, while the running version keeps serving the wrong-but-current answer. The backfill reaches the running version's offsets on all six partitions and continues live beside it. The operator confirms the cutover; both versions pause, settle at offsets (8,102,441,...) on every partition, and the name moves. The dashboard's next read shows the corrected totals for every window, including last week's. Its subscription ends with PRV-4019 and the SDK resubscribes from a fresh snapshot. The sink — a JDBC table — is sent one batch: the rows that differ. Had the corrected numbers looked wrong, a rollback within the hour would have put the old version back the same way.

What is refused. Replacing a name that shares its computation with another name (PRV-8003); backfilling a stream nothing is bound to, a stream that cannot be replayed, or one whose positions do not order its records (PRV-4018); the design's adaptive backfill options, because there is no probe of the store's latency to adapt to (PRV-4018); and a replacement by a principal who may not administer the name — reading its status included, because a candidate's SQL describes a query the caller may not be allowed to read. One interaction is open: replacing a query over a `postgres-cdc` stream is not refused, and the candidate's backfill probably contends for the running version's replication slot (finding CDCREPL-1, open, not reproduced).

8 Lanes: isolation against memory

The previous four sections are about correctness at seams. This one is about the substrate the computations run on, because the choice it offers — how many queries share a lane — trades isolation for memory, and the trade is a property the operator is entitled to see stated.

8.1 Confinement

A lane owns an inbox, an arena and its operators' state, and exactly one thread drives it at a time: a fixed pool of one runner thread per core steps many lanes in turn, and a lane belongs to one runner thread from the moment it is hosted until it is removed. That is the whole concurrency model: a lane's arena, operator state and inbox cursors have no synchronisation because one thread touches them.

Proposition 8.1 (Single writer). *At any moment at most one thread executes a given lane's operators, and a lane's state is never read or written by another lane's thread.*

Runs: `LaneRunnerTest: manyLanesRunOnFewThreadsAndEachSeesItsOwnRows`. Confinement being the correctness model is why the design rejected a work-stealing pool of queries as tasks (ADR-027), which would touch state from whichever thread picked the task up.

8.2 Own, shared, auto and dedicated

By default, under `pravaha.lane.multiplex.enabled = auto`, the first 64 queries on a node (`auto-from`) each own a lane, and every registration after them shares one of a fixed set of

shared lanes — one per available processor by default, each carrying at most 300 queries. A registration is placed on the least loaded shared lane below the ceiling, lowest index on a tie; what the query reads does not matter. `true` shares from the first query and `false` never shares. A single query can ask for a lane of its own whatever the mode, with `WITH (lane = 'dedicated')`, which is journalled so a restart keeps it.

Proposition 8.2 (Placement does not change answers). *A query’s answer is the same whether it runs on a lane of its own or shares a lane with others, whatever they read.*

Argument. On a shared lane each row carries a route in its header. Every hosted input has a private route, so what a query is fed alone reaches that query alone; a reader shared by several queries on the lane writes one copy under a shared route, and every query on the lane that reads it is handed that one copy. The multiplexer dispatches by route, never by stream name, so two queries over one stream on one lane do not each count the other’s rows — the defect recorded as LANE-1 before routes existed. $\square$

Runs: `SharedLaneIngestPropertyTest: queriesSharingALanesIngestAnswerExactlyAsOnLanesOfTheirOwn`; `SharedLanePlacementTest: twoQueriesOverOneStreamShareALaneAndEachCountsOnlyItsOwnRows`, `queriesOverDifferentStreamsShareALaneAndKeepTheirOwnAnswers`, `eachRegistrationLandsOnTheLeastLoadedLane`; `LaneMultiplexerTest: aPrivateRouteReachesItsOwnQueryAloneAndASharedRouteReachesEveryListener`, `aJoinsTwoInputsArriveOnTheirOwnInputsThroughOneInbox`.

Proposition 8.3 (Sharing never admits fewer queries). *A registration that fits on no shared lane is given a lane of its own rather than refused. Turning sharing on therefore never makes a node accept fewer queries than it does with sharing off.*

Runs: `SharedLanePlacementTest: aRegistrationNoLaneWillTakeGetsALaneOfItsOwnAndADropFreesTheRoom`, `withMultiplexingOffEveryQueryHasALaneOfItsOwn`. The fallback’s cost is visible as `pravaha_lane_own_queries`.

Proposition 8.4 (Fate-sharing). *(i) A query on a lane of its own fails alone: a pipeline that throws drops its lane, and other lanes, including those sharing its runner thread, keep running.*

(ii) Queries on one shared lane share its fate: a pipeline that throws — including one refused for its state with PRV-4001 — drops the lane and every query on it.

(iii) Queries on one shared lane share its backpressure: a query slow for its share slows the lane’s drain for all of them, and a full inbox holds every writer feeding the lane; a shared reader, which writes a row to all its lanes or none, then stalls every query it feeds on any lane. A lane that frees no cell for thirty seconds fails its feed with PRV-3002 rather than hanging it.

(iv) Nothing is dropped for anybody in (iii): a slow query becomes a slow read of its source.

In part: (i) runs: `LaneRunnerTest: oneLaneFailingDoesNotStopTheOthersSharingItsThread`; `LaneGroupTest: aFailedLaneLeavesItsSiblingsRunning`; `LaneDeathVisibilityTest: aLaneThatDiesOnTheFeedPathIsAskableAbout`. (ii) and (iii) are stated in the operations documentation and follow from a lane being the unit of failure and of buffering; we found no test that fails one query on a shared lane and asserts that its neighbours stop, and none that measures one query’s slowness on its neighbours’ latency. They are argued, not tested. The per-query backpressure metrics — how often and how long a writer found no room, recorded against the query whose writer it was — exist so that “this query is the limit” can be told apart from “this query is queued behind a neighbour”.

In plain terms. A lane of its own is a room of its own: if the query in it falls over, nobody else notices, and if it is slow, it is slow alone. A shared lane is a shared room: much cheaper, but one

query falling over takes the room down, and one slow query slows everyone in it. The default gives the first sixty-four queries rooms of their own, where isolation is cheap, and shares after that, where a megabyte per idle query is what limits the node. A query that matters more than its megabyte can ask for its own room.

Placement	Off-heap per query	Failure	Backpressure
Own lane	an inbox: 1,024 KiB idle at the defaults, 1,328 KiB once rows move	alone	alone
Shared lane	its share of one inbox per lane; 1,000 queries on 8 lanes hold 8 inboxes	with every query on the lane	with every query on the lane, and every lane its shared reader feeds
Dedicated	as own lane, kept across restarts	alone	alone

The per-query figures are `NodeScaleTest`'s as reported in the operations documentation; the 1,000-query figure is `SharedLaneDensityTest`'s (`aThousandQueriesOverOneSourceShareEightLanesAndOneCopyOfEachRowPerLane`), which also asserts that the thousand queries' shared reader writes each source row into the eight lanes once each — eight copies, where lanes of their own write a thousand. Section 12 gives the conditions.

8.3 Moving a query is a replacement

Placements are never rebalanced automatically: a lane's cost is not knowable at registration, and a scheme that guessed would have to move a running lane to correct itself, which confinement forbids. Slots freed on shared lanes by drops, and headroom under `auto-from`, go to new registrations. Moving a running query between a shared lane and a lane of its own is instead a blue/green replacement with the SQL unchanged: `CREATE OR REPLACE ... WITH (lane = 'dedicated')` moves it to a dedicated lane and `lane = 'shared'` back. An administrator's *rebalance* does this in bulk: it hands the room under `auto-from` to shared queries, oldest first, one at a time, each moved by a replacement with `lane = 'own'` (a lane of its own, not pinned: a restart re-replaces it by the mode). Queries answering to several names or already being replaced are skipped; a move a replacement refuses is reported and the rest go on.

Corollary 8.5 (Rebalancing preserves answers). *Each move of a rebalance has the guarantees of Theorem 7.3: the query's readers see its answer before the move and after it, over the same input at the seam, with nothing lost or counted twice; its subscribers are told the view was replaced.*

Proof. A move is a replacement whose candidate has the same SQL, so Theorem 7.3 applies with *A* and *B* computing the same query; clause (ii) then says *B*'s view equals *A*'s at the seam. □

Runs: `LaneRebalanceTest: roomLeftByADroppedQueryGoesToTheOldestSharedOneOnlyWhenAskedAndIts AnswerSurvives; DedicatedLaneTest: createOrReplaceMovesARunningQueryOntoItsOwnLaneAndBackWith TheSameSql, aRestartBringsADedicatedQueryBackOnItsOwnLaneAndCompactionKeepsIt, theSame ComputationAlreadyOnASharedLaneIsRefusedAndOnItsOwnLaneIsJoined.`

That the one administrative operation which changes where a query runs reduces to the one operation that already had a lossless seam is, we think, the right shape: there is no second mechanism for moving state between lanes to get wrong, and the cost — a backfill, one query at a time — is the cost of the replacement, paid visibly.

9 Serving the answer

A maintained answer is only useful if reading it is cheap and reading it is *right*. The engine serves each view from the process that maintains it — there is no second database for the result to live in — over Arrow Flight SQL, a REST endpoint, and the PostgreSQL wire protocol. All three run the same reader (`ViewQuery`), so what follows holds for all three.

9.1 Consistency of a read

A read declares one of four modes. **Consistent**, the default, reads the last commit (Proposition 4.2) and reports how far behind the latest applied change that is. **Latest** reads the committed map with the pending overlay on top and says when it may include uncommitted work. **AtLeast(*f*)** waits, for a bounded time, until the view has committed through frontier *f* and then reads it; it gives up with a named error rather than hanging when a source is idle. **AsOf(*f*)** is refused: a view holds the present, and answering an audit question about four o'clock with the value at five would be the worst possible response.

Runs: `ServedViewTest: staleneessComesBackWithTheAnswer`, `anAtLeastReadReturnsOnceTheFrontierArrives`, `anAtLeastReadGivesUpRatherThanHanging`, `anAsOfReadIsRefusedRatherThanAnsweredWithThePresent`, `aMissStillSaysWhatItWasAMissAsOf`.

9.2 Access paths change cost, never answers

A read's **WHERE** clause decides how its rows are found. The whole key by equality is one hash probe on every view, because a view already is a map by that key. The key's leading columns by equality and its last column between bounds walk an ordered index over the key's last column, built the first time a range read needs it and maintained by every commit after (ADR-049). A non-key column by equality or **IN**, when the registration declared **INDEX (column)**, probes an equality index once per value (ADR-055). Anything else — a partial key, an **OR** across columns, **<>** — scans the committed rows.

Proposition 9.1 (Access-path invariance). *For every view state and every predicate φ , the rows a read returns are $\{r \in V : \varphi(r)\}$ whichever access path `ViewAccessPath` chooses.*

Proof. `ViewAccessPath` returns a candidate set *C* and does not rewrite the plan; the filter φ still runs over *C*. It is written to return only sets with $C \supseteq \{r \in V : \varphi(r)\}$: a conjunct it does not recognise is skipped, which widens *C*; an **OR** or a **NOT** at the top pins nothing; a literal that does not fit the column's width makes it give up and scan. Filtering a superset by φ yields exactly the rows satisfying φ . $\square$

Runs: `ViewIndexEquivalenceTest: everyGeneratedPredicateAnswersTheSameWhicheverPathItTakes` and `theSameHoldsWhileTheViewIsChangingUnderTheReads`, which load the same rows into two views differing only in their key, so that the paths apply to one and not the other, and put generated predicates to both; `ViewIndexTest: theIndexAndTheViewAgreeAfterAnyRunOfChanges`, `theIndexIsMaintainedUnderTheSameLockAsTheCommit`.

9.3 An equality index over a column outside the key

ADR-049 declined this index and named why: a row's non-key values change under such an index, so every update is a delete and an insert in the index, and the entry to delete has to be found from the row's *previous* values — which a Z-set retraction may not carry, or may carry wrongly. An index that disagrees with its view is a wrong answer with a confident face. ADR-055 builds it with one rule.

Proposition 9.2 (The index is a function of the view). *Let $\iota_c(V)$ map each value x of column c to the set of keys of rows in V whose c is x (non-null values only). After every commit, eviction and restore, the equality index on c equals $\iota_c(V)$ for the view’s committed contents V .*

Proof. The index is never told what changed. In `ServedView.commit`, for each key in the overlay, the index is handed the row the view currently holds for that key (`visible.get(key)`) and removes the key from that row’s value’s bucket, then files the new row, in the same critical section as the view’s own update. The retraction that caused the change is not consulted, so a retraction with a wrong value, no value, or a value from a row the view never held cannot mislead it. Eviction removes the entry filed under the evicted row; a restore clears the index and rebuilds it from the restored rows before the monitor is released. By induction over commits the index equals ι_c of the visible map. $\square$

Runs: `SecondaryIndexTest: theIndexAnswersExactlyWhatTheScanAnswersWhateverHappensToTheView` (generated inserts, retractions carrying arbitrary values, key moves, evictions and restores, with every probe compared to the scan after every commit), `anUpdateThatChangesTheIndexedColumnMovesTheRowInTheSameCommit`, `aRestoreRebuildsTheIndexBeforeTheFirstRead`; `EqualityIndexTest: aRemovalFiledUnderTheWrongValueRemovesNothing`; `SecondaryIndexRegistryTest: aReadByTheIndexedColumnIsTheScansAnswerThroughRetractionsAndARestart`, `aReplacementCarriesTheIndexToTheNewVersionByNameAndRefusesToChangeIt`. ADR-055 records that seeding the obvious bug — removing under the incoming row’s value — fails both.

Proposition 9.3 (When a stored-equality probe is sound). *Let $=_{\text{sql}}$ be the filter’s equality on a column’s values and $=_{\text{st}}$ equality of the stored representation after conversion to the column’s stored class. A probe for literal x that returns the rows whose stored value is $=_{\text{st}}$ -equal to x returns a superset of the rows satisfying $c =_{\text{sql}} x$, for every x and every view, if and only if $y =_{\text{sql}} x \Rightarrow y =_{\text{st}} x$ for all values x, y .*

Proof. If the implication holds, every row with $c =_{\text{sql}} x$ has $c =_{\text{st}} x$ and is returned. Conversely, if some $y =_{\text{sql}} x$ has $y \neq_{\text{st}} x$, the view holding one row with $c = y$ and the literal x make the probe miss a row the filter keeps; since the filter runs only over what the probe returns (Proposition 9.1), the read loses it. $\square$

Runs: the refusal it justifies: `SecondaryIndexRegistryTest: anIndexThisEngineCannotKeepIsRefusedByName`, with code PRV-2074 (`SQL_INDEX_UNUSABLE`).

The proposition is the whole reason for the refusals. For `FLOAT`, `0.0 =sql -0.0` but the stored values differ; for `DECIMAL`, `1.0 =sql 1.00` but their stored representations differ; for `BYTES`, arrays compare by identity. Each is refused at registration, against the planned output. Whole numbers, temporal types, `BOOLEAN` and text — whose equality is exact here, which is also why `<` on text is refused — satisfy the implication and are indexable. (`NaN`, equal to nothing under $=_{\text{sql}}$, is harmless to a superset.) The ordered index needs the stronger property that the stored order is a total order agreeing with the filter’s comparisons, so it additionally refuses text (it would need a collation) and `BOOLEAN` (PRV-2073).

In plain terms. An index is allowed to make a read faster or slower. It is never allowed to change the answer, and two rules make sure. First, whatever the index finds, the filter still runs over it, so the index only has to find *at least* the right rows. Second, the index is kept from the rows the view actually holds, never from what a change message claims the old row was. A column is refused an index when two values the filter calls equal would be stored as different — `1.0` and `1.00`, or `0.0` and `-0.0` — because then the index could miss a row the filter wanted.

An index costs one entry per row whose value is non-null, referencing the row the view already holds rather than copying it; a view keeps at most four; so an index is bounded by the ceiling that bounds the view. It is on the heap and does not spill, because the view does not either (the memory-mapped tier of ADR-044 is for operator state). The declaration is an

allocation, written to the registry journal in the same write as the registration, so a restart cannot bring the query back without it.

10 Security, identity and tenancy, as they affect the model

Security enters this paper only where it changes what a computation is or who may share one. Three places.

Authorization is at the engine, on derived rows. A view is derived data the store has never seen: there is no record in the source whose permissions correspond to “u42’s gold-tier total for the window ending at 12:05”, and a change feed read once and shared by many queries cannot be authorized per principal at the source without reading it once per principal. So authorization is enforced by the engine, in the read path itself (`ViewQuery`), not in a transport (ADR-031). A decision is allow, allow with a row filter, or deny, and every decision is audited.

Proposition 10.1 (The soundness rule for filters from outside a query). *A filter supplied from outside a query — a principal’s row filter, a subscriber’s tap filter, a continuous-query parameter — can be applied to the query’s view if and only if the view carries every column the filter names. Otherwise it is refused (row filter, PRV-7003; tap filter, PRV-8002) or, for a parameter, the binding forks its own computation.*

Proof. If the view carries the columns, filtering the view’s rows by the predicate selects exactly the rows derived from input rows that satisfy it, since those columns pass through unchanged. If the query aggregated a named column away, each view row mixes input rows the filter is meant to separate, and no function of the view row can unmix them; applying the filter anyway would either leak the mixed total or drop rows arbitrarily. $\square$

Runs: `ViewQueryAuthorizationTest: aRowFilterSurvivesAnAggregate, aRowFilterCannotBeUndoneByThe CallersOwnWhereClause, aRowFilterIsAppliedWhetherOrNotTheQueryMentionsItsColumn; Subscription Test: aFilterOnAColumnTheViewDoesNotHaveIsRefused`. Row filters are, by ADR-031’s own status line, programmatic only: the rule and the plan injection are built and tested, and no server configuration can yet supply a policy that returns a row filter.

Sharing is by fingerprint, and the fingerprint includes entitlement. Two registrations share one computation when their *fingerprints* are equal, and the fingerprint is a hash of a canonical encoding of: the normalised plan *after* security predicates have been injected into it; the tenant; the key columns in their order; and the retention (`QueryFingerprint`).

Proposition 10.2 (Sharing never crosses an entitlement or a tenant). *If two registrations differ in the row filters applied to them, their tenant, their key columns or their retention, they are two computations. Identical questions from principals of one tenant with the same entitlements are one computation.*

Proof. Each differing component is written into the canonical encoding. The tenant is length-prefixed, so no tenant string can be read as a different tenant followed by a row-filter clause (an unprefixed tenant “t\nsecurity:p” would otherwise encode as tenant t with row filter p). Distinct encodings hash to distinct fingerprints except with the hash’s collision probability. $\square$

Runs: `TenancyTest: identicalSqlInTwoTenantsIsTwoComputations, identicalSqlWithinATenantStill SharesOneComputation, aTenantNameThatSmugglesANewlineDoesNotHashAsARowFilter; SharingIdentity Test: twoKeyingsOfTheSameSqlAreTwoComputations, keyOrderIsPartOfTheIdentity, twoRetentionsOfThe SameSqlAreTwoComputations, identicalQuestionsStillShareOneComputation`. ADR-025’s status line still says the key columns are not in the fingerprint; the code and these tests are the current state.

Sharing is conservative, and deliberately so. Normalisation goes as far as the planner’s canonical form and no further, so `WHERE a > 0 AND b > 5` and `WHERE b > 5 AND a > 0` are two computations. Sharing too little costs a second copy of the state; sharing too much would be two principals reading each other’s rows. Where the two risks meet the design takes the first.

A tenant owns names and state, and does not own a lane. Under ADR-050 a tenant owns the names it registers, the computations behind them, and the keys their views hold, and is admitted by quota: a registration that would exceed the tenant’s query count or view-key count is refused by name (`PRV-8020`, `PRV-8021`) before a sink is opened or a feed started, and a replacement from another tenant is refused (`TenancyTest: aTenantAtItsQueryQuotaIsRefusedByNameAndAnotherTenantIsNot`, `aReplacementFromAnotherTenantIsRefusedByName`). A tenant does *not* scope lanes or CPU. Shared lanes are shared across tenants, so Proposition 8.4(ii) and (iii) cross tenant boundaries: one tenant’s query that fails on a shared lane takes down the other tenants’ queries on that lane, and one tenant’s slow query slows them. Per-tenant CPU scheduling is not built. An operator who needs isolation between tenants has, today, `lane = 'dedicated'` and sharing turned off.

Identity. Since ADR-052 the engine is its own identity authority: it keeps users, Argon2id password hashes, API keys (shown once, scoped, expiring, rotatable) and sessions, and one verifier serves every transport. What matters for the model is only its output: a **Principal** carrying an id, a tenant, roles and claims, which is what the policy, the fingerprint and the quotas read. Multi-factor authentication and single sign-on were dropped by the owner’s decision.

11 The register: what carries each claim

A guarantee documented and unenforced decays into an assertion within a few releases, and thereafter misleads. The discipline we hold this paper to is that each claim either names the test that asserts it, says which part runs, or says plainly that it is argued. The register below is kept in the paper and not in the system, which is weaker than an executable register would be; we record that rather than present the table as sufficient.

Claim	State	Where, or why not
Z-sets form a group (Proposition 3.2)	runs	<code>ZSetTest</code> generated properties
Linear lift, DISTINCT lift, recomputation, $D \circ I$ (Propositions 3.4 to 3.6 and 3.8)	algebra only	<code>IncrementalOracleTest</code> , <code>ZSetTest</code> ; <code>pravaha-algebra</code> is imported by no running module
Bilinear join rule (Theorem 3.9)	algebra only	<code>IncrementalJoinTest</code> ; the runtime’s <code>SymmetricHashJoin</code> is not compared with it
Chains of views (Corollary 3.10)	algebra only	<code>IncrementalOracleTest</code>
Runtime operators satisfy (1) (Observation 3.11)	in part	differential tests between implementations and against hand-computed answers; no oracle comparison
Presence is net weight (Proposition 3.14)	runs	<code>ServedViewTest</code>
The keyed view is the Z-set (Proposition 3.15)	argued	conditional on key-functional input; finding <code>VIEWW-1</code> open
Watermark monotone, idle exclusion (Proposition 3.17)	runs	<code>WatermarkTrackerTest</code>

Claim	State	Where, or why not
Corrections consolidate; too-late is counted (Proposition 3.19)	runs	LateDataTest, StreamAllowedLatenessTest
Commit atomic to readers (Proposition 4.2)	runs	ServedViewTest
Audience fixed at first batch (Proposition 4.3)	runs	SnapshotHandoffTest
Plain subscribe plus read is gapful (Observation 4.4)	runs	SubscribeFromSnapshotTest
Snapshot then every commit once (Theorem 4.6)	runs; (S3) argued	SnapshotHandoffTest, SubscribeFromSnapshotTest, EmbeddedSnapshotSubscriptionTest, SnapshotSubscriberBehindTest
Freeze and exact marker (Lemmas 5.3 and 5.4)	runs	ControlTaskBarrierTest, AlignedCheckpointBarrierTest
Checkpoint is one cut; state exactly once (Theorem 5.5)	runs	AlignedCheckpointBarrierTest, CheckpointRecoveryTest, AggregateRestartTest, StateRestoreTest
No checkpoint across the exchange (Remark 5.7)	argued	refusal in QueryExecution; unreachable, no pipeline sends on the exchange
Exactly-once to a transactional sink (Theorem 5.9)	runs	TransactionalSinkDeliveryTest, KafkaSinkBrokerTest, KafkaSinkRegistrationTest, DeltaSinkRegistrationTest
Weakest link (Proposition 5.10)	runs	CapabilitiesTest, SinkGuaranteeListingTest
Waiting member attached at its position (Lemma 6.5)	runs	ExactSharingTest
Exact-seam sharing (Theorem 6.6)	in part	ExactSharingTest over a test source; filesystem bounded read tested; Kafka's poll Before untested directly
The splice (Lemma 7.1)	runs	OffsetSplicedReaderTest
Lossless cutover (Theorem 7.3)	runs; (iii) argued	QueryReplacementTest, BackfillSpliceEquivalenceTest, ReplacementSinkHandoverTest; no test races reads against the swap
Rollback window (Proposition 7.4)	runs	QueryReplacementTest, ReplacementSinkHandoverTest
Replacement survives restart (Proposition 7.5)	runs	ReplacementRestartTest
Single writer (Proposition 8.1)	runs	LaneRunnerTest
Placement does not change answers (Proposition 8.2)	runs	SharedLaneIngestPropertyTest, SharedLanePlacementTest, LaneMultiplexerTest
Sharing never admits fewer (Proposition 8.3)	runs	SharedLanePlacementTest
Fate-sharing (Proposition 8.4)	in part	(i) runs; (ii)–(iv) argued
Rebalance preserves answers (Corollary 8.5)	runs	LaneRebalanceTest, DedicatedLaneTest
Access-path invariance (Proposition 9.1)	runs	ViewIndexEquivalenceTest, ViewIndexTest
Index is a function of the view (Proposition 9.2)	runs	SecondaryIndexTest, EqualityIndexTest, SecondaryIndexRegistryTest
Stored-equality soundness (Proposition 9.3)	proved; refusal runs	anIndexThisEngineCannotKeepIsRefusedByName
Filter soundness rule (Proposition 10.1)	runs	ViewQueryAuthorizationTest, SubscriptionTest; row filters programmatic only
Fingerprint separates entitlements (Proposition 10.2)	runs	TenancyTest, SharingIdentityTest

Claim	State	Where, or why not
Cross-view consistent read	not in PRAVAHA	no read returns several views at one frontier

Of thirty-five rows, twenty-three run as stated; two run with one clause argued; one is proved here, with the refusal it justifies running; three run in part; three — covering seven results — hold of the reference algebra only; two are argued; and one construction is absent. The rows that matter most to a reader deciding whether to trust the engine are the algebra-only rows: the design rests on DBSP’s laws and the tests show those laws of a module the engine does not execute. The seam theorems, which are this paper’s own contribution, are better carried than the algebra they sit on, and that asymmetry is the first thing Section 14 proposes to fix.

12 Evaluation: what has been measured, and under what conditions

This section reports only figures recorded in the repository, names the file and test each comes from, and states the conditions. It makes no comparison with another system, because none has been run. The theorems above are about correctness; almost nothing below is, and Section 12.7 lists the correctness-adjacent costs nobody has measured.

12.1 The requirement, and the machine

The product requirement is on the order of *a thousand rows a second* (ADR-042): the engine maintains answers to registered questions rather than moving bulk data. The design’s original gate figures — 1.2 million rows a second per lane for a filter-and-project profile (Profile A), 350,000 for a windowed aggregate (Profile B), and at least 90% of linear scaling from one lane to eight — are kept unchanged as gate criteria, and ADR-042 records that restating the requirement did not close them.

There is no reference hardware. Every figure below was taken on one development machine: an AMD Ryzen AI 9 HX 370 laptop part, 12 physical cores of two designs (Zen 5 and Zen 5c) with SMT2 (24 logical), 61 GiB of memory, OpenJDK 21.0.12 with G1. On 2026-09-20 it was running other build agents and containers throughout, with load averages between 5.8 and 77.8 on 24 logical processors, printed beside every figure. The evidence pack (`docs/gates/measured-2026-09-20/README.md`) says in its first section that the numbers are a floor for one machine on one afternoon and not the engine’s capability, and we repeat that here.

12.2 Throughput gates

Measurement	Figure	Conditions and source
Profile A, one lane, warm pool (512 rows, in L2)	best of five 29.9–31.0 M rows/s over three runs	load 6.5–9.2; ProfileAGateIT
Profile A, through the registry and a served view, cold pool (34.7 MiB)	best 5.9–18.5 M rows/s; worst single pass 1,043,974 rows/s	load 23.5–77.0; the worst pass is the one below the 1.2 M target
Profile B, windowed aggregate, 100,000 Zipf keys	best 2.47–2.85 M rows/s; worst of fifteen passes 1.10 M	load 7.2–8.1; ProfileBGateIT
Scaling, one lane to eight (2026-09-20)	28–42 % of linear, eight runs	JaCoCo agent attached (found later); ProfileAGateIT
Scaling, one lane to eight (2026-09-26)	33–46 % of linear	agent removed; load 1.1–4.4
The machine’s own scaling, 16 independent threads	51–55 % of linear	machineScalingReference arm

The per-lane throughput criteria of Profiles A and B are reached on this machine; the scaling criterion is not. The 2026-09-26 re-measurement found that every earlier scaling run carried a coverage agent whose shared probe arrays made eight lanes contend on the same cache lines; that JFR attributed half of each lane’s samples to decoding a string column for every row in a text comparison, since fixed; and that the machine itself gives 51–55 % of linear for sixteen threads that share nothing. Against that ceiling the engine keeps 60–90 % of what the machine gives, but the gate is stated against linear and is recorded as not reached. Nothing here is evidence that the software scales, or that it does not.

12.3 Generated against interpreted code

A registered query’s filter and projection have run generated code since 2026-09-26. The pipeline alone, fed in a loop on one thread, runs 106–118 million rows a second generated against 60–66 million interpreted, 1.7 $\times$. End to end through a lane the two are level, 0.95 $\times$ to 1.13 $\times$ by best pass, because one producer thread copying every row into the inbox is then the bound. The README’s “roughly 10 $\times$ ” is a different measurement (**ProfileABenchmark**: the generated batch loop against interpreted operators over one batch, with no pipeline, arena or sink around either), and the evidence pack says so.

12.4 Nexmark coverage

Win condition W5 — Nexmark’s queries against Flink SQL on the same hardware — has not been run and cannot be here: there is no Flink deployment, no quiet machine and no Nexmark generator in the repository. What was measured is how many of the 23 published queries the engine runs at all: 5 on 2026-09-20 and 12 on 2026-09-26 (q0, q1, q2, q3, q7, q8, q9, q18, q19, q20, q21, q22; **NexmarkCoverageIT** pins the list). The rest are refused for missing SQL, not for speed: unwindowed grouping (five queries, refused for unbounded state), a window aggregate over a row frame, session windows, processing time, a user-defined function, and a partitioned file sink. Throughputs of the twelve that run were recorded once, at load 2.53, from the harness’s own generator: from 65,705 rows/s (q20) to 6,934,430 (q2). The pack says these “are not Nexmark numbers and must never be presented as any”, because the data is not Nexmark’s and the missing queries are the expensive ones; we do not present them as such.

12.5 What a query costs

Measurement	Figure	Source
Off-heap per idle query on its own lane	1,024 KiB (the inbox)	NodeScaleTest, OPERATIONS.md
Off-heap per active query on its own lane	1,328 KiB	NodeScaleTest
Platform threads added by 1,000 queries, 24 cores	+24	ThousandQueryTest, EXECUTION_MODEL.md §6
Off-heap for 1,000 queries, default / advised sizing	1,000 MiB / 61 MiB	ThousandQueryTest
Registration, per query, 1,000 queries	3.7 ms	ThousandQueryTest
Before lane multiplexing (ADR-036)	1.00 thread and ~5 MiB per query	NodeScaleTest, recorded as the “before” figures
1,000 queries over one source, shared lanes	8 lanes, 8 copies of each row	SharedLaneDensityTest
Four queries over one Aerospike set, shared reader	1.0 scans/s in total, where it was 1.0 each	ADR-036, against a real single-node cluster

These are counts rather than rates — bytes, threads, copies — and are stable across runs in a way the throughput figures are not; ADR-036 notes that a CPU figure from this engine is quotable only from a quiet machine, and that a process-wide CPU counter inside a full build reported a *negative* per-source cost.

12.6 Subscribers, state beyond memory, and instrumentation

Subscribers. `SubscriptionIngestCostTest` (ADR-026) measured one query, 100,000 distinct keys, a commit every 250 rows, at load 10.36. In process, with a consumer that returns at once, ingest was flat in subscriber count: 680,474 rows/s with none and 663,204 with twenty. Over Flight, with subscribers connected and never reading, it fell from 536,768 with none to 130,656 with twenty. The difference is the carrier: Arrow Flight serialises each batch per subscriber on that subscriber’s call thread, and has no call that hands an already-serialised batch to a second listener. A node’s subscriber capacity is therefore bounded by serialisation and linear in subscribers; ADR-026 records that an earlier “encode once, write N times” claim was never true.

State larger than memory. ADR-044 measured the memory-mapped overflow tier twice (`SpillTierMeasurementIT`, `SpillBeyondRamMeasurementIT`, the second under a cgroup memory cap with swap off). While the page cache holds the spilled state, throughput stays within about $2\times$ of RAM; once the state exceeds what the node can cache, a random lookup costs page faults and throughput falls to about 1,000–2,500 operations a second on the machine’s NVMe, against 160,000–540,000 cached. The tier is off by default and documented as survival, not capacity.

Instrumentation. Per-operator counters and sampled self time cost about 8% of a narrow query’s throughput (`OperatorMetricsOverheadIT`), so they are compiled in only when switched on.

12.7 What has not been measured

The costs of the mechanisms this paper is about are, with one exception, unmeasured:

- how long a checkpoint’s freeze holds the sources, and its distribution under load;

- how long an exact-seam catch-up takes to converge on a moving seam, and how much reading the exact sharing of ADR-054 saves over a reader per query for Kafka (`ExactSharingTest` counts readers; it does not time them);
- how long a replacement’s alignment takes, how often it retries, and how long backfills take at realistic history sizes;
- what a slow or failing query on a shared lane does to its neighbours’ latency;
- end-to-end latency from a source write to a visible commit, and commit latency beyond the exact mean the metrics report;
- recovery time after a crash as a function of state size;
- anything with more than one node.

The exception is the cost of subscribers, above. We regard the absence of seam-cost measurements as the most important gap in this section: a guarantee whose price is unknown is a guarantee an operator cannot plan around.

13 Related work

Incremental view maintenance. Maintaining a query’s result under changes to its inputs is a long-studied problem [2, 3]. Griffin and Libkin gave the treatment of views with duplicates by counting [4]; Koch’s ring of databases and DBToaster’s higher-order deltas showed how far delta rules compose [5, 6]; and the signed multiplicities PRAVAHA uses are Green, Karvounarakis and Tannen’s $\mathbb{Z}$ -relations [10, 11]. DBSP [7] recasts incremental maintenance as a small algebra over streams of $\mathbb{Z}$ -sets with integration, differentiation and a mechanical lift, covering recursion and a rich SQL; PRAVAHA’s design follows it and its reference module implements the fragment Section 3 states. We claim nothing new in this area, and Observation 3.11 records that the running engine is not yet held to the algebra by an oracle.

Differential dataflow and Materialize. Differential dataflow [8], built on timely dataflow [9], maintains collections as differences indexed by partially ordered times, which also supports iteration. Materialize [25] builds a SQL database on it that maintains views incrementally and serves them, with reads that can be consistent across views at a shared timestamp and a subscription that can begin with a snapshot of the current contents. PRAVAHA offers the snapshot subscription (Theorem 4.6) but not cross-view consistency; it is single-node where Materialize is distributed; and it is embeddable and reads the operator’s existing stores directly. We are not aware of a published statement of Materialize’s internal subscription hand-over comparable to Theorem 4.6, and do not claim one is absent.

Apache Flink. Flink [14] made aligned barrier checkpoints — a form of Chandy and Lamport’s snapshot [12] adapted to dataflows [13] — and two-phase commit sinks the standard for exactly-once stream processing [15]. PRAVAHA’s checkpoint is the same idea on one node with one lane per registered query, which is why it can freeze the sources rather than align barriers across channels, and why it refuses the multi-lane exchange case it has not built. Flink SQL represents updates as changelogs of insert, update-before, update-after and delete rows, which downstream operators interpret; PRAVAHA represents every change as a weighted row and lets addition do the work, following DBSP. Flink upgrades a job by stopping it with a savepoint and restarting the new job from that state, which is fast and requires the new job’s state to be compatible with the old; PRAVAHA’s replacement instead recomputes the new version’s state from replayable history and cuts over at a position, which works across arbitrary SQL changes

and costs a replay and a retention requirement on the source. The two are complementary rather than competing answers.

Kafka Streams and ksqlDB. Kafka Streams achieves exactly-once processing with Kafka’s transactions, keeping state in local stores backed by changelog topics, and exposes that state to interactive queries; Wang et al. describe the design and its consistency and completeness trade-offs [22]. ksqlDB [23] provides SQL on top, with push queries for changes and pull queries against materialised state. Both are built around Kafka as source, state log and sink; PRAVAHA treats Kafka as one source and one sink among several and stages Kafka output per checkpoint because a restarted producer cannot commit a transaction prepared by its predecessor.

RisingWave. RisingWave [26] is a PostgreSQL-compatible streaming database that maintains materialised views incrementally with barrier-based checkpoints and keeps its state in shared object storage, separating compute from storage across nodes. It shares PRAVAHA’s premise that the answer should be served where it is maintained; it differs in being distributed and storage-disaggregated, where PRAVAHA is one embeddable node with off-heap state and an optional memory-mapped overflow tier.

Noria. Noria [24] maintains materialised views for read-heavy web applications as a partially stateful dataflow, filling missing state on demand with upqueries, and adapts the running dataflow when queries are added, reusing existing operators. Its sharing of dataflow between queries and its in-place evolution are close in spirit to PRAVAHA’s fingerprint sharing and replacement; the mechanisms differ — Noria changes the dataflow in place, PRAVAHA runs a second computation and cuts over at a position.

Streams, time and SQL. CQL gave continuous queries a semantics in terms of streams and relations [21]. Watermarks, allowed lateness and retraction of refined results are the Dataflow model’s [16, 17], and Begoli et al. compare watermark semantics across systems [18]. PRAVAHA plans SQL with Apache Calcite [20], whose streaming extensions Begoli et al. describe [19]. Its windowed aggregates use slicing, in the line of panes [30] and Scotty [31]; its generated pipelines follow Neumann’s data-centric compilation [27].

Shared scans. Sharing one scan among several queries, and letting a late query join a scan already in progress, is well established in databases [28, 29]. For a set semantics a late joiner can simply wrap around. For an ordered, exactly-once log it cannot, and the contribution of Section 6 is the contract under which it can still join exactly: an order on positions and a read that stops at a position.

Property-based testing. Much of the evidence in Section 11 is generated-input testing in the QuickCheck tradition [32], with jqwik on the JVM, and in several cases the test is shown to fail on a seeded version of the bug it guards against.

14 Limitations, what is not built, and future work

14.1 What the implementation does not establish

The algebra is not an oracle for the engine. The largest gap. The laws of Section 3 are tested of `pravaha-algebra`, which the engine does not execute (Observation 3.11). The engine’s operators are tested differentially and against hand-computed answers. A defect common to the interpreted and generated paths, or to a pushed-down and a local evaluation, would pass every equivalence test.

One node. PRAVAHA runs on one node. Membership, fenced partition leases (proved against a real ZooKeeper ensemble), and rebalance and handoff are built as libraries and consumed by no node; a node asked to start in partitioned mode refuses (PRV-9002). The owner has put cluster mode on hold. Every theorem in this paper is a single-node theorem, and a query moved between nodes would be a new seam this paper does not treat — though Theorem 7.3 suggests the shape its answer should take.

Open findings against stated invariants. VIEWWW-1 (Proposition 3.15): a retraction that leaves a key’s net weight positive may leave the retracted row’s values as the key’s row. CDCREPL-1: a replacement over a `postgres-cdc` stream probably contends for the running version’s replication slot. Neither is reproduced; both are recorded in the repository’s findings register as open and not blocking. The windowed aggregate’s 128-bit slice digest (W8-14) is a bounded, recorded departure from exact keys.

Partial evidence for the seam theorems. Kafka’s bounded read has no broker-backed test with transaction markers at the seam; fate-sharing on a shared lane is argued; no test races reads against a cutover’s catalogue swap.

What the exchange and the lanes do not do. A registered query runs on one lane; the exchange between lanes is not cut by a checkpoint, and such a checkpoint is refused. A tenant does not own a lane, so a shared lane’s fate is shared across tenants, and per-tenant CPU scheduling is not built.

Scope of exact sharing. Only the filesystem (not followed) and Kafka declare ordered positions and bounded reads. Delta and JDBC wait on a proof that their positions are totally ordered. Change-data-capture sources keep a reader per query because a shared slot could be confirmed only to the slowest member’s durable position.

SQL. Session windows, window aggregates over row frames, `ORDER BY`, set operations, subqueries other than the temporal join, and unwindowed grouping over a stream are refused; outer joins between streams need a time bound. Twelve of Nexmark’s twenty-three queries run. `MIN` and `MAX` are not retractable incrementally and are never pre-combined at a source.

Performance claims. The scaling gate is not reached on the only machine available; there is no reference hardware and no head-to-head comparison with any other system. None of the throughput figures in Section 12 is a capability.

14.2 How the account could be shown wrong

We would regard each of the following as a refutation of a specific claim, not a defect report to be triaged: a generated sequence of changes on which a runtime operator’s output diverges from the reference algebra’s (Observation 3.11); a snapshot subscriber whose copy diverges from the view without being ended by PRV-6105 (Theorem 4.6); a restore whose answer counts a record twice or misses one, over a replayable source, without a refusal (Theorem 5.5); a Kafka partition with a control record at the seam on which a late joiner receives a record twice or not at all (Theorem 6.6); a read of a replaced name that returns a row of neither version’s view at the seam (Theorem 7.3); and an index probe that returns a different answer from the scan (Propositions 9.1 and 9.2).

14.3 Future work

1. **Hold the engine to the algebra.** Run the runtime’s operators, interpreted and generated, and `pravaha-algebra`’s lifts on the same generated changes and compare, as `Incremental OracleTest` does for the reference lifts. This would move three rows of the register from “algebra only” to “runs”.
2. **An executable register.** One file stating every claim of Section 11 with the test that carries it, and a test that fails when a named test disappears or the paper and the file disagree.
3. **A broker-backed seam test** for Kafka with transaction markers at and around the seam, and a fate-sharing test on a shared lane.
4. **Measure the seams:** freeze duration, catch-up convergence, alignment retries and backfill time, so an operator can plan for the guarantees’ cost.
5. **Close VIEWW-1** with a Z-set property test on the served view, and decide CDCREPL-1.
6. **Cut the exchange:** forward the marker along each ring and align on it, which would admit multi-lane registered queries into Theorem 5.5.
7. **Cross-view reads at one frontier**, which the reference `Frontier` describes and the serving layer does not provide.
8. **Cluster mode**, in which moving a query between nodes should be a replacement across nodes, cut over at a position, by the same argument as Corollary 8.5.

15 Conclusion

A continuous query is an old idea whose arithmetic is well understood. What makes a system built on it trustworthy is not the arithmetic but the hand-overs: the moments where a subscriber’s copy meets the live stream, where restored state meets the replay, where a late query meets a shared reader, and where a new version of a query meets the old one. We have argued that these are one problem, and that the answer is the same each time: make the seam a position rather than a moment, read positions and state together under a freeze, and refuse when equality at the seam cannot be shown. Stated that way, each of the four has a short proof, and each proof names the assumptions it borrows from the sources and sinks around the engine.

We have also tried to be exact about the distance between the design and what the tests show. The seam theorems are carried by tests that exercise the seams, several of them shown to fail on the defect they guard against. The algebra underneath them is carried by tests of a module the engine does not run. That asymmetry is the most useful thing this paper can report to someone deciding whether to rely on the engine, and it is the first thing to fix.

The slogan the project uses is *ask once, answer always*. The theorems in this paper are what “always” has to mean for it to be worth saying: across a subscriber joining, a restart, a shared reader and a change of question, the answer neither skips a record nor counts one twice.

References

- [1] D. Terry, D. Goldberg, D. Nichols, B. Oki. Continuous queries over append-only databases. In *Proc. ACM SIGMOD*, pages 321–330, 1992.
- [2] J. A. Blakeley, P.-Å. Larson, F. W. Tompa. Efficiently updating materialized views. In *Proc. ACM SIGMOD*, pages 61–71, 1986.

- [3] A. Gupta, I. S. Mumick. Maintenance of materialized views: problems, techniques, and applications. *IEEE Data Engineering Bulletin*, 18(2):3–18, 1995.
- [4] T. Griffin, L. Libkin. Incremental maintenance of views with duplicates. In *Proc. ACM SIGMOD*, pages 328–339, 1995.
- [5] C. Koch. Incremental query evaluation in a ring of databases. In *Proc. ACM PODS*, pages 87–98, 2010.
- [6] Y. Ahmad, O. Kennedy, C. Koch, M. Nikolic. DBToaster: higher-order delta processing for dynamic, frequently fresh views. *PVLDB*, 5(10):968–979, 2012.
- [7] M. Budiu, T. Chajed, F. McSherry, L. Ryzhyk, V. Tannen. DBSP: automatic incremental view maintenance for rich query languages. *PVLDB*, 16(7):1601–1614, 2023.
- [8] F. McSherry, D. G. Murray, R. Isaacs, M. Isard. Differential dataflow. In *Proc. CIDR*, 2013.
- [9] D. G. Murray, F. McSherry, R. Isaacs, M. Isard, P. Barham, M. Abadi. Naiad: a timely dataflow system. In *Proc. ACM SOSP*, pages 439–455, 2013.
- [10] T. J. Green, G. Karvounarakis, V. Tannen. Provenance semirings. In *Proc. ACM PODS*, pages 31–40, 2007.
- [11] T. J. Green, Z. G. Ives, V. Tannen. Reconcilable differences. In *Proc. ICDT*, pages 212–224, 2009.
- [12] K. M. Chandy, L. Lamport. Distributed snapshots: determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985.
- [13] P. Carbone, G. Fóra, S. Ewen, S. Haridi, K. Tzoumas. Lightweight asynchronous snapshots for distributed dataflows. arXiv:1506.08603, 2015.
- [14] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, K. Tzoumas. Apache Flink: stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, 38(4):28–38, 2015.
- [15] P. Carbone, S. Ewen, G. Fóra, S. Haridi, S. Richter, K. Tzoumas. State management in Apache Flink: consistent stateful distributed stream processing. *PVLDB*, 10(12):1718–1729, 2017.
- [16] T. Akidau, A. Balikov, K. Bekiroğlu, S. Chernyak, J. Haberman, R. Lax, S. McVeety, D. Mills, P. Nordstrom, S. Whittle. MillWheel: fault-tolerant stream processing at internet scale. *PVLDB*, 6(11):1033–1044, 2013.
- [17] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, S. Whittle. The Dataflow Model: a practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *PVLDB*, 8(12):1792–1803, 2015.
- [18] E. Begoli, T. Akidau, S. Chernyak, F. Hueske, K. Knight, K. Knowles, D. Mills, D. Sotolongo. Watermarks in stream processing systems: semantics and comparative analysis of Apache Flink and Google Cloud Dataflow. *PVLDB*, 14(12):3135–3147, 2021.
- [19] E. Begoli, T. Akidau, F. Hueske, J. Hyde, K. Knight, K. Knowles. One SQL to rule them all: an efficient and syntactically idiomatic approach to management of streams and tables. In *Proc. ACM SIGMOD*, pages 1757–1772, 2019.
- [20] E. Begoli, J. Camacho-Rodríguez, J. Hyde, M. J. Mior, D. Lemire. Apache Calcite: a foundational framework for optimized query processing over heterogeneous data sources. In *Proc. ACM SIGMOD*, pages 221–230, 2018.
- [21] A. Arasu, S. Babu, J. Widom. The CQL continuous query language: semantic foundations and query execution. *The VLDB Journal*, 15(2):121–142, 2006.

- [22] G. Wang, L. Chen, A. Dikshit, J. Gustafson, B. Chen, M. J. Sax, J. Roesler, S. Blee-Goldman, B. Cadonna, A. Mehta, V. Madan, J. Rao. Consistency and completeness: rethinking distributed stream processing in Apache Kafka. In *Proc. ACM SIGMOD*, pages 2602–2613, 2021.
- [23] H. Jafarpour, R. Desai, D. Guy. KSQL: streaming SQL engine for Apache Kafka. In *Proc. EDBT*, pages 524–533, 2019.
- [24] J. Gjengset, M. Schwarzkopf, J. Behrens, L. T. Araújo, M. Ek, E. Kohler, M. F. Kaashoek, R. Morris. Noria: dynamic, partially-stateful data-flow for high-performance web applications. In *Proc. USENIX OSDI*, pages 213–231, 2018.
- [25] Materialize, Inc. Materialize documentation. <https://materialize.com/docs/>, accessed 2026.
- [26] RisingWave Labs. RisingWave documentation. <https://docs.risingwave.com/>, accessed 2026.
- [27] T. Neumann. Efficiently compiling efficient query plans for modern hardware. *PVLDB*, 4(9):539–550, 2011.
- [28] S. Harizopoulos, V. Shkapenyuk, A. Ailamaki. QPipe: a simultaneously pipelined relational query engine. In *Proc. ACM SIGMOD*, pages 383–394, 2005.
- [29] M. Zukowski, S. Héman, N. Nes, P. Boncz. Cooperative scans: dynamic bandwidth sharing in a DBMS. In *Proc. VLDB*, pages 723–734, 2007.
- [30] J. Li, D. Maier, K. Tufte, V. Papadimos, P. A. Tucker. No pane, no gain: efficient evaluation of sliding-window aggregates over data streams. *SIGMOD Record*, 34(1):39–44, 2005.
- [31] J. Traub, P. M. Grulich, A. Rodríguez Cuéllar, S. Breß, A. Katsifodimos, T. Rabl, V. Markl. Scotty: efficient window aggregation for out-of-order stream processing. In *Proc. IEEE ICDE*, pages 1300–1303, 2018.
- [32] K. Claessen, J. Hughes. QuickCheck: a lightweight tool for random testing of Haskell programs. In *Proc. ACM ICFP*, pages 268–279, 2000.

A Where the code is

For a reader checking the markers, the classes named in the paper live as follows (module, then package under `com.ash.messaging.pravaha`).

Module	Classes named in this paper
<code>pravaha-algebra</code>	<code>ZSet</code> , <code>Lift</code> , <code>IncrementalQuery</code> , <code>Query</code> , <code>IncrementalJoin</code> , <code>Integrate</code> , <code>Differentiate</code> , <code>Frontier</code>
<code>pravaha-api</code>	<code>StreamSourcePlugin</code> , <code>PartitionReader</code> , <code>BoundedPartitionReader</code> , <code>OrderedPositions</code> , <code>StreamSinkPlugin</code> , <code>DeliveryGuarantee</code>
<code>pravaha-runtime</code>	<code>QueryExecution</code> , <code>IngestPump</code> , <code>WindowedAggregate</code> , <code>SlicedAggregateState</code> , <code>SymmetricHashJoin</code> , <code>KeyedAggregate</code> , <code>TopNRanking</code> , <code>LookupJoin</code> , <code>WatermarkTracker</code> , <code>LaneRunner</code> , <code>LaneMultiplexer</code>
<code>pravaha-serving</code>	<code>ServedView</code> , <code>ViewSink</code> , <code>ViewQuery</code> , <code>ViewAccessPath</code> , <code>EqualityIndex</code> , <code>Consistency</code>
<code>pravaha-backfill</code>	<code>OffsetSplicedReader</code> , <code>SplicedReader</code>
<code>pravaha-bindings</code>	<code>SharedPartitionFeed</code> , <code>SharedSourceGroup</code>
<code>pravaha-registry</code>	<code>QueryRegistry</code> , <code>QueryReplacements</code> , <code>QueryFingerprint</code> , <code>SharedLanes</code> , <code>LaneRebalance</code> , <code>TenantQuotas</code> , <code>RegistryJournal</code>

The decision records cited (ADR-008, 013, 015, 025, 026, 027, 029, 031, 036, 042, 044, 046, 049, 050, 052, 054, 055) are in `docs/adr/`; the findings cited (VIEW-1, VIEWW-1, CDCREPL-1, SUB-1, STRM-11, LANE-1, LANE-6, SRC-3, W8-14) are in `docs/qa/FINDINGS.md`; the measurements are in `docs/gates/measured-2026-09-20/README.md` and the ADRs named beside each figure.

Copyright © 2026 Ashutosh Sinha. This paper is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International Licence (CC BY-NC-ND 4.0); see `LICENSE` in this directory. The PRAVAHA software it describes is proprietary and is not covered by that licence.